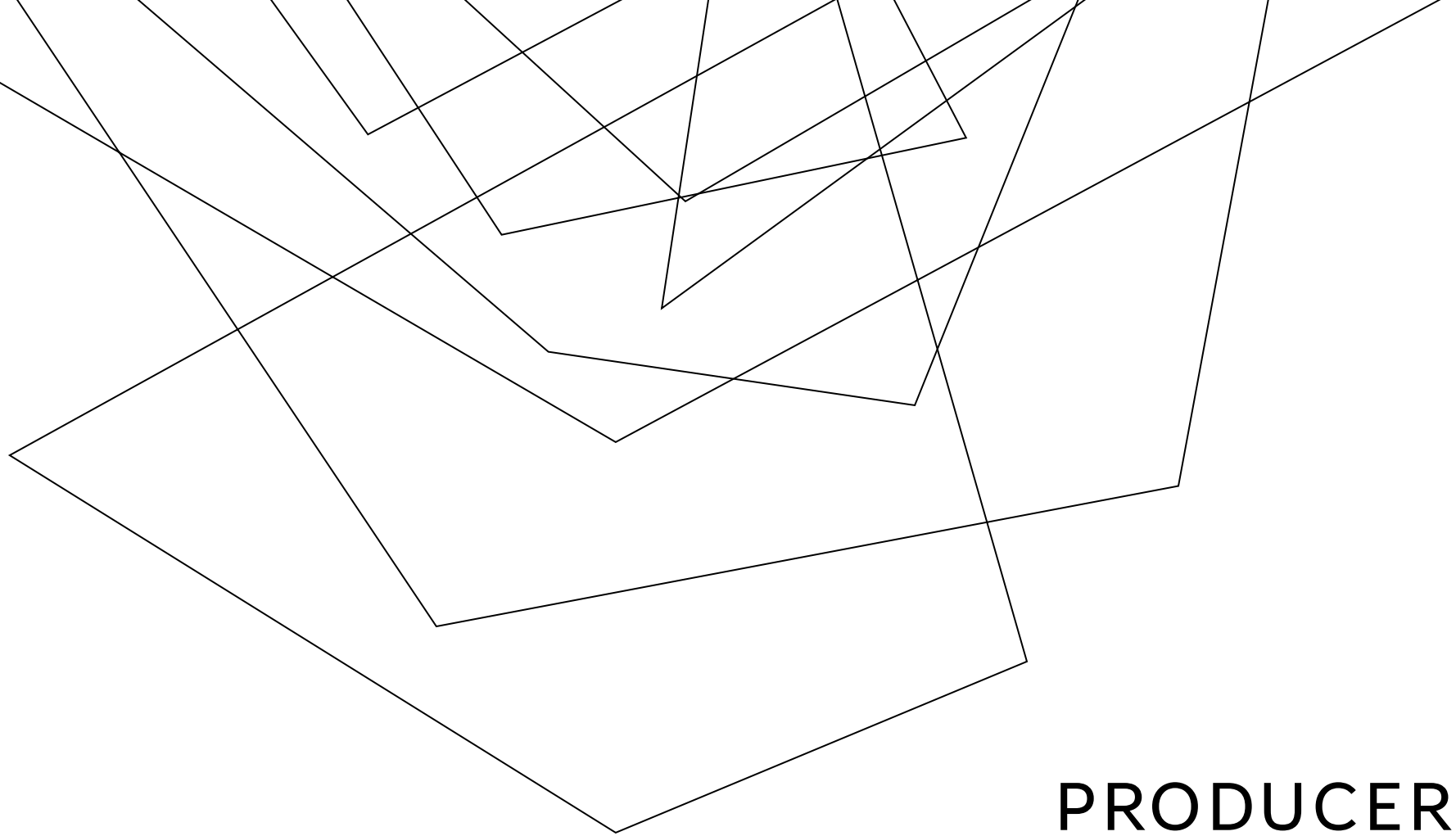
ERROR HANDLING AND RELIABILITY

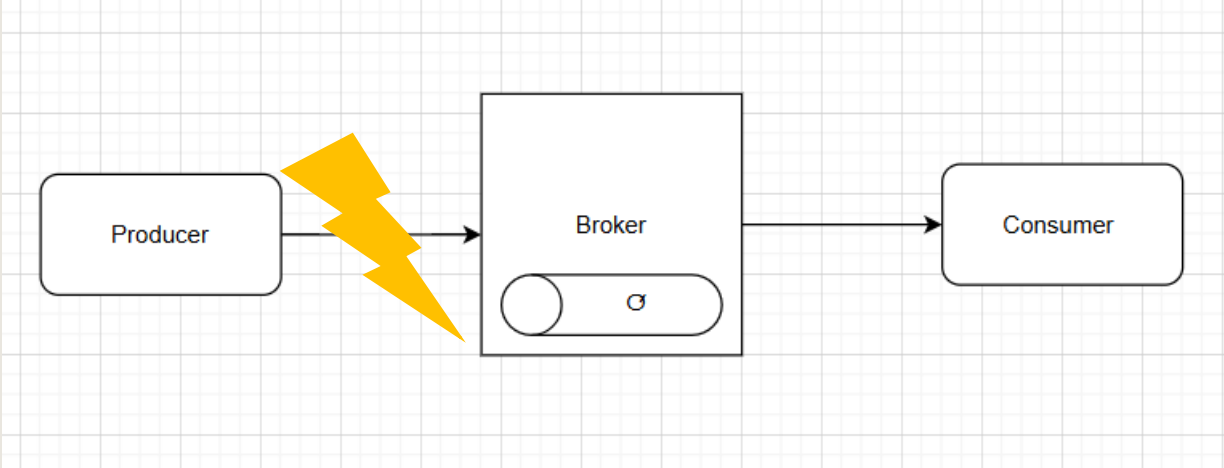
SESSION 8 - LEARN STRATEGIES FOR HANDLING ERRORS AND ENSURING RELIABILITY IN MESSAGE PROCESSING



PRODUCER-SIDE ERROR HANDLING

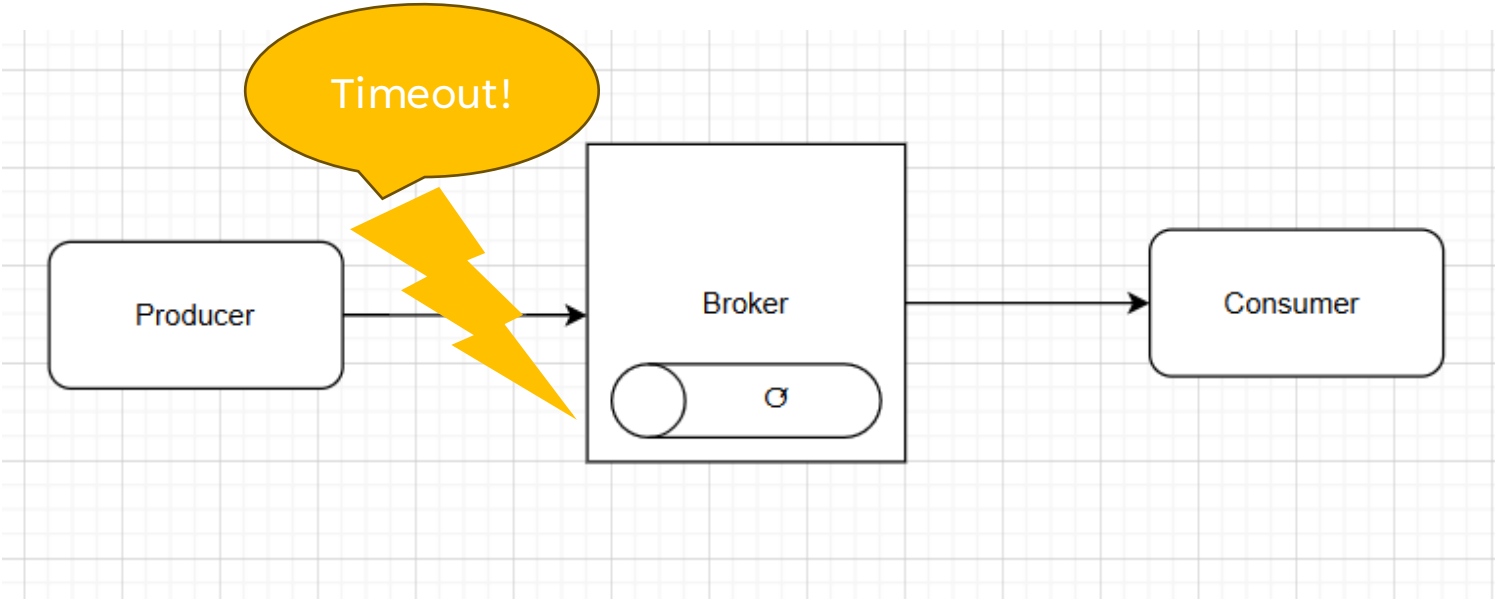
CONNECTION FAILED

The Producer failed establishing connection



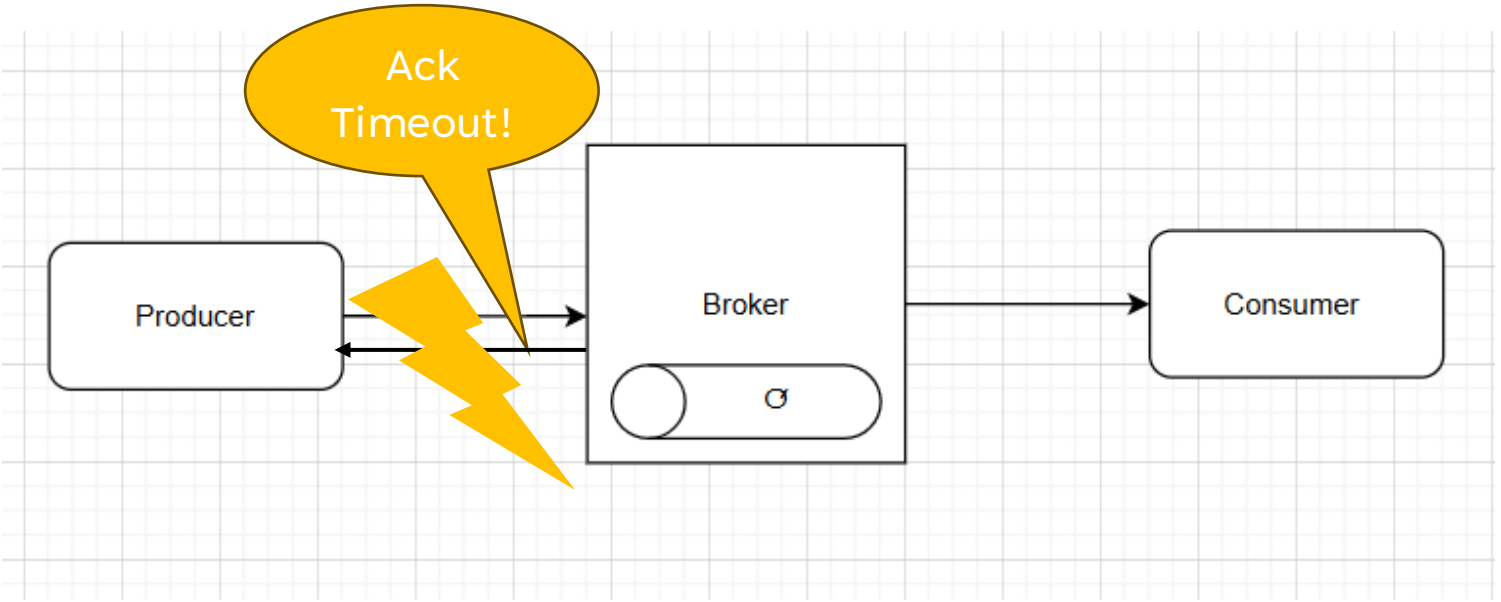
CONNECTION FAILED - Description

• **Connection Timeouts**: When the producer can't establish a connection to the broker within a certain time frame.



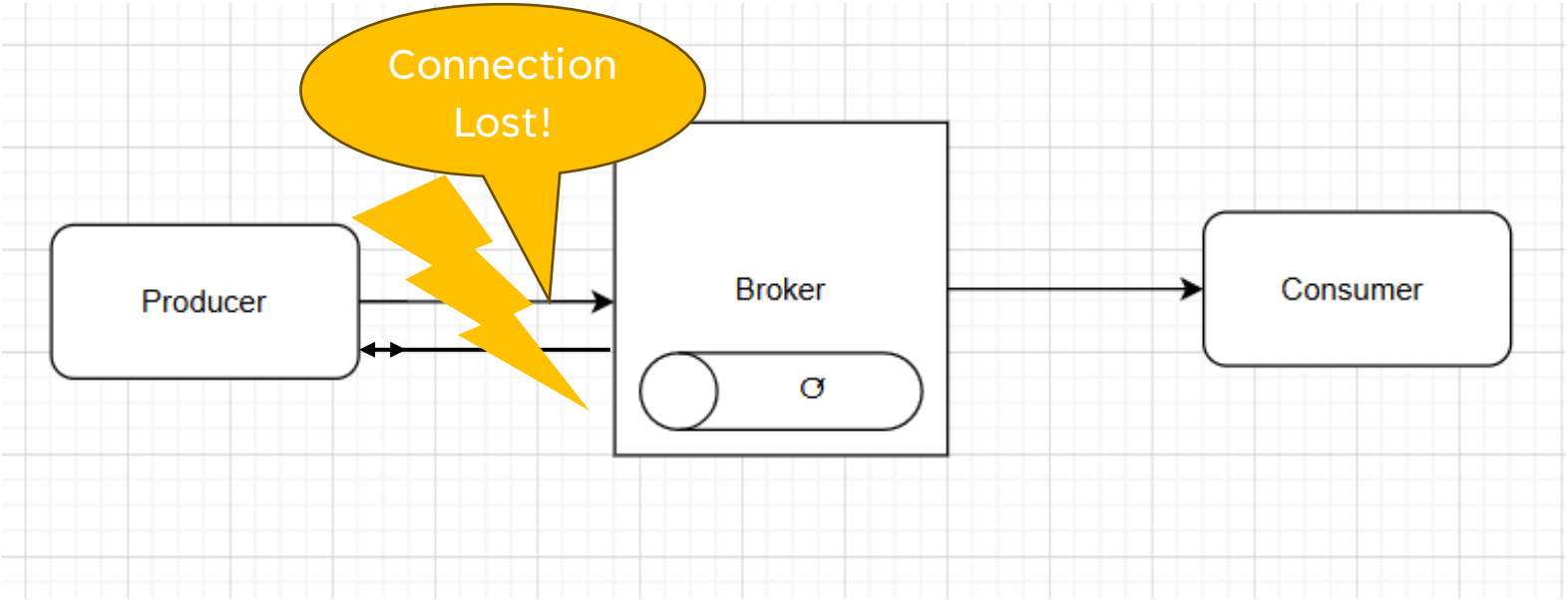
CONNECTION FAILED - Description

• **Write Timeouts**: The connection is established, but the data isn't acknowledged by the broker within a specified period.



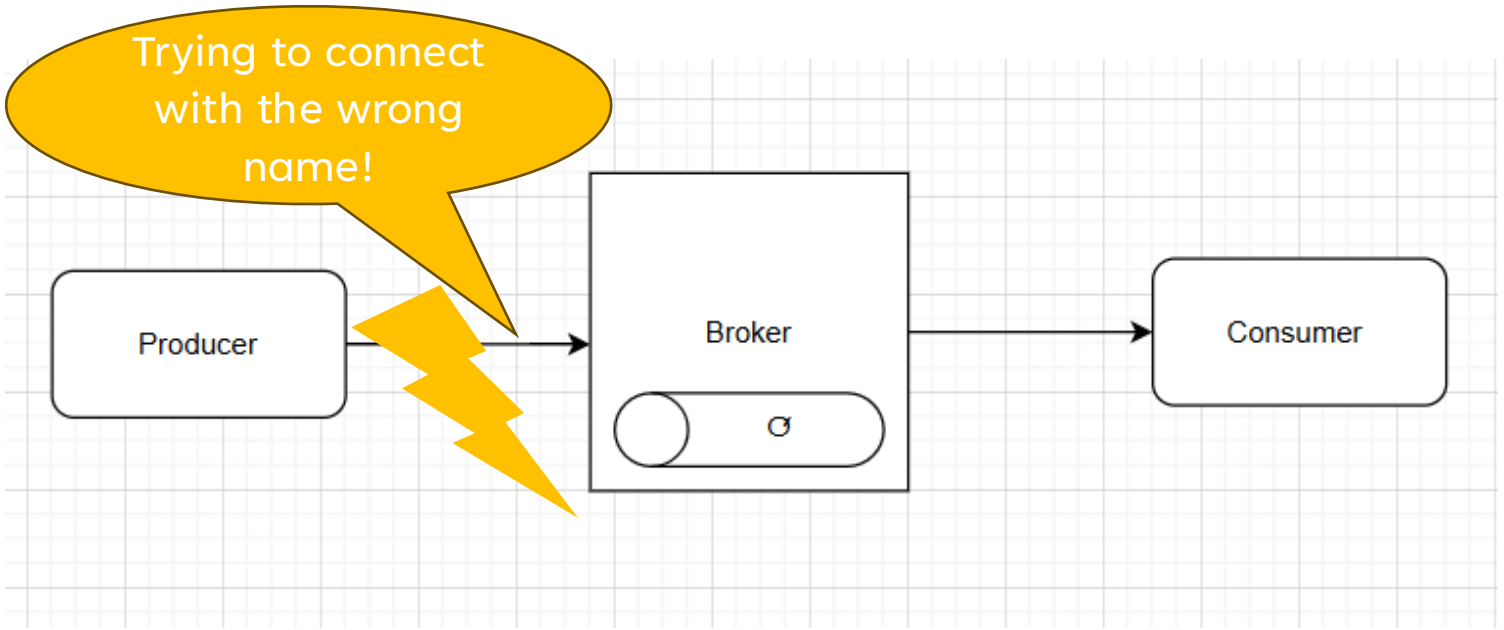
CONNECTION FAILED - Description

• **Connection Loss**: A previously established connection is lost unexpectedly during message transfer.



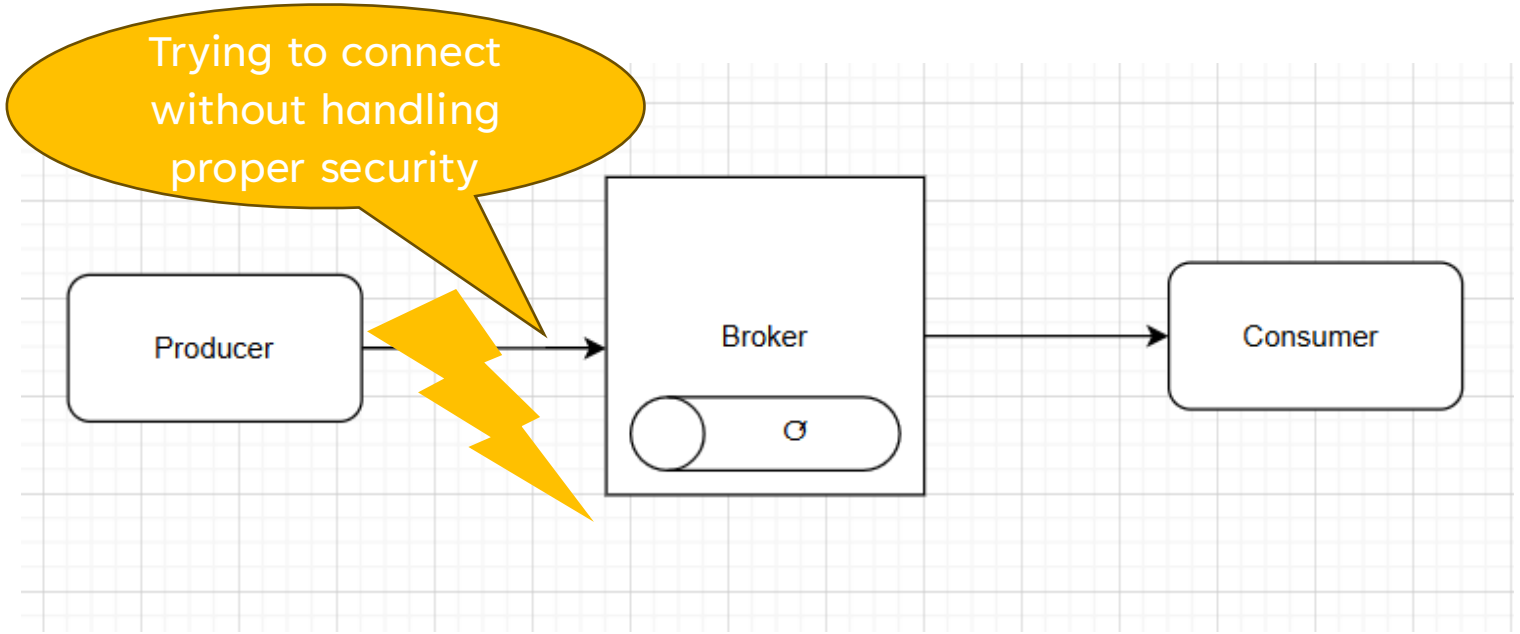
CONNECTION FAILED - Description

• **DNS Failures**: If the broker's hostname can't be resolved, it can prevent the producer from reaching the broker.



CONNECTION FAILED - Description

• **SSL/TLS Handshake Failures**: In secure connections, SSL/TLS issues can prevent connections from being established.



CONNECTION FAILED - How to detect

- **Error Codes and Exceptions**: Most messaging libraries throw specific exceptions or return error codes for network-related issues

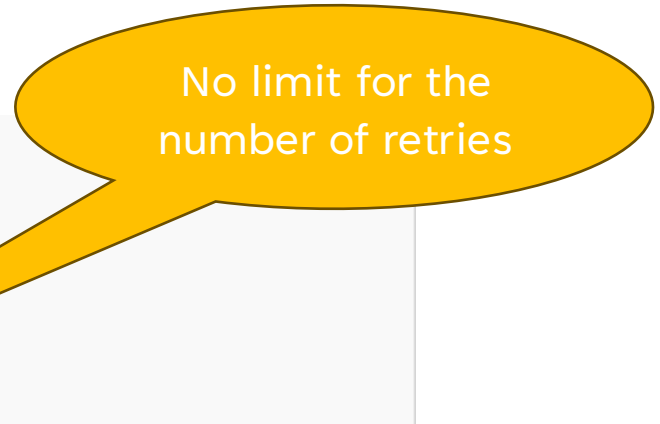
- **Connection Status Monitoring**: Monitor the status of connections to the broker in real-time. Many libraries, including Kafka and RabbitMQ clients, provide **callback** mechanisms or events for connection status changes.

- **Retry Failures and Alerting**: Configure a retry threshold that, when exceeded, triggers an alert indicating repeated network issues.

CONNECTION FAILED – How to Solve in RabbitMQ

- **Automatic Connection Recovery**: Use the AutomaticRecoveryEnabled flag to automatically recover connections when they drop.

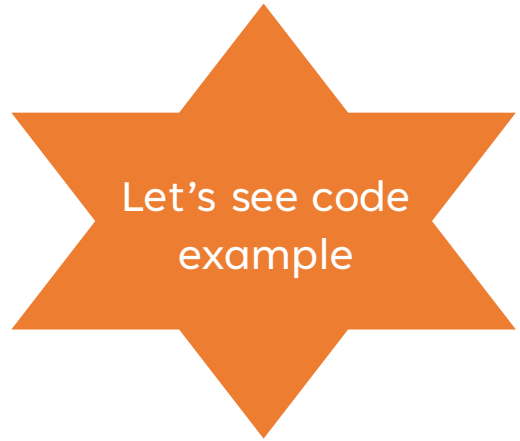
```
var factory = new ConnectionFactory()
{
    HostName = "localhost",
    UserName = "guest",
    Password = "guest",
    AutomaticRecoveryEnabled = true, // Enable automatic recovery
    NetworkRecoveryInterval = TimeSpan.FromSeconds(10) // Time interval between reconnection
};
```



CONNECTION FAILED – How to Solve in RabbitMQ

- **Reconnection Logic**: Implement logic to attempt reconnecting to RabbitMQ if the connection is lost. You can use exponential backoff or jittered retry mechanisms to avoid overwhelming the broker.

- **Handling Disconnected States**: Ensure that the producer buffers or retries messages while the connection is down to avoid message loss.



HANDLE NETWORK AND RECONNECT RABBITMQ

Best Practice:

- Enable **automatic connection recovery** to handle temporary network issues.
- Implement **reconnection logic** and **retry policies** to handle longer-lasting failures.

```
var connectionFactory = new ConnectionFactory
{
    HostName = "localhost",
    AutomaticRecoveryEnabled = true,
    NetworkRecoveryInterval = TimeSpan.FromSeconds(5) // Retry every 5 seconds
};

using (var connection = connectionFactory.CreateConnection())
using (var channel = connection.CreateModel())
{
    // Produce messages
}
```

CONNECTION FAILED – How to Solve in Kafka

Kafka provides callbacks to handle asynchronous responses and errors.

Using OnError and OnLog callbacks, you can monitor and log network errors, helping in diagnostics.

```
var producer = new ProducerBuilder<string, string>(producerConfig)
    .SetErrorHandler((_, e) =>
        Console.WriteLine($"Kafka Error: {e.Reason}"))   // Logs any error
    .SetLogHandler((_, log) =>
        Console.WriteLine($"Log: {log.Message}"))        // Logs operat
    .Build();
```

Invokes on every error from KafkaClient

Invokes on every log from KafkaClient

FAILED PUBLISH- How to Solve in RabbitMQ

RabbitMQ uses the AMQP protocol, which supports fault-tolerant patterns for handling network issues.

1. Automatic Retries with Backoff:
   - Implement retry logic manually by catching network exceptions and re-sending the message.

   - Backoff Strategy: Use exponential backoff to prevent flooding the network and define a maximum retry count to avoid indefinite attempts.

```
var retryCount = 0;
var maxRetries = 5;
var backoff = TimeSpan.FromSeconds(1);

while (retryCount < maxRetries)
{
    try
    {
        channel.BasicPublish(exchange, routingKey, properties, messageBody);
        break; // Successful send
    }
    catch (BrokerUnreachableException)
    {
        retryCount++;
        Thread.Sleep(backoff);
        backoff = backoff.Add(TimeSpan.FromSeconds(1)); // Exponential backoff
    }
}
```

FAILED PUBLISH- How to Solve in RabbitMQ

RabbitMQ uses the AMQP protocol, which supports fault-tolerant patterns for handling network issues.

1. Automatic Retries with Backoff:
   - Implement retry logic manually by catching network exceptions and re-sending the message.

   - Backoff Strategy: Use exponential backoff to prevent flooding the network and define a maximum retry count to avoid indefinite attempts.

```
var retryCount = 0;
var maxRetries = 5;
var backoff = TimeSpan.FromSeconds(1);

while (retryCount < maxRetries)
{
    try
    {
        channel.BasicPublish(exchange, routingKey, properties, messageBody);
        break; // Successful send
    }
    catch (BrokerUnreachableException)
    {
        retryCount++;
        Thread.Sleep(backoff);
        backoff = backoff.Add(TimeSpan.FromSeconds(1)); // Exponential backoff
    }
}
```

FAILED PUBLISH- How to Solve in Kafka

Kafka producers offer various configurations to handle produce errors and ensure message delivery.

1. **Retries and Backoff**:
   Kafka producers have a built-in retry mechanism to handle transient errors (e.g., network interruptions, broker unavailability).
   • **retries**: Controls how many times the producer should retry sending a message if it fails.
   • **retry.backoff.ms**: Specifies the time interval between retries. Increasing this value allows the system to have more time to recover from temporary issues.

FAILED PUBLISH- How to Solve in Kafka

Best Practice: Set the retries to a sufficiently high number and retry.backoff.ms to a reasonable value (e.g., 100ms) to handle transient network or broker issues without overwhelming the system with retries.

```
var producerConfig = new ProducerConfig
{
    BootstrapServers = "localhost:9092",
    EnableIdempotence = true,           // Ensures no duplicate message
    Acks = Acks.All,                    // Requires acknowledgment from all replicas
    Retries = int.MaxValue,             // Allows indefinite retries
    RetryBackoffMs = 500
};
```

FAILED PUBLISH- How to Solve in Kafka

2. **Idempotence**: Kafka supports **idempotent producers**, which ensure that even if a producer sends the same message multiple times (due to retries), it will be written only once to the broker. This prevents **duplicate messages** in the event of retries.

• **enable.idempotence=true**: This guarantees that messages are written to the broker exactly once in the presence of retries, ensuring **exactly-once** delivery semantics.

FAILED PUBLISH- How to Solve in Kafka

Best Practice: Always enable idempotence (enable.idempotence=true) for critical applications to avoid duplicate messages during retry scenarios.

```
var producerConfig = new ProducerConfig
{
    BootstrapServers = "localhost:9092",
    EnableIdempotence = true,           // Ensures no duplicate message
    Acks = Acks.All,                    // Requires acknowledgment from all replicas
    Retries = int.MaxValue,             // Allows indefinite retries
    RetryBackoffMs = 500
};
```

HANDLE MESSAGE PERSISTENCE FOR DURABLE DELIVERY

To ensure that messages are not lost in case of broker failure, it is important to use **persistent messages.**

- In **Kafka** the messages are not deleted from the topic and has robust system.

- In **Rabbit** we need to publish them to **durable queues**.

HANDLE MESSAGE PERSISTENCE FOR DURABLE DELIVERY IN RABBITMQ

Best Practice:

•Set props.Persistent = true for critical messages.

• Ensure that the target queues are **durable** by creating them with the durable flag set to true.

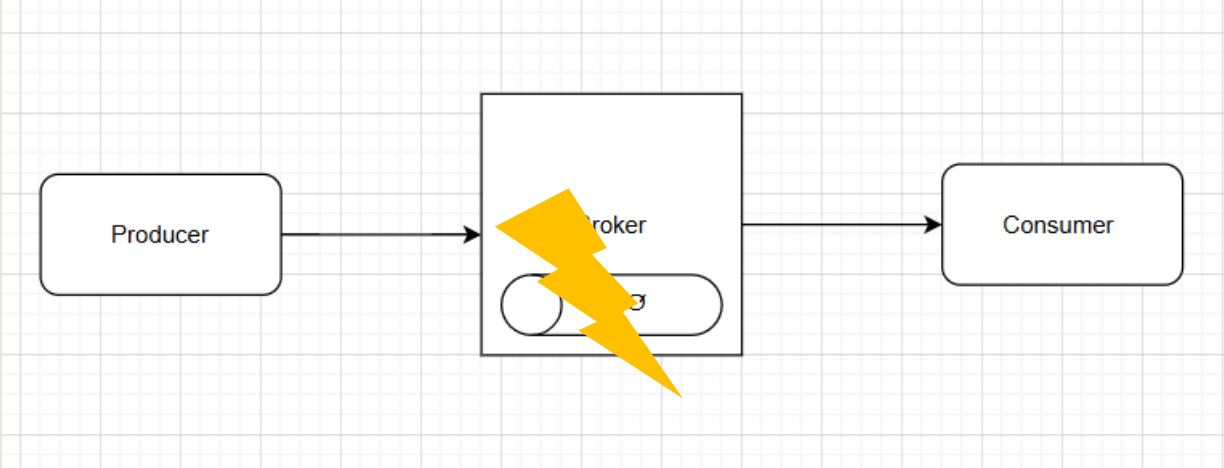
```
var props = channel.CreateBasicProperties();
props.Persistent = true; // Enable persistence

channel.QueueDeclare(queue: "durable_queue",
                     durable: true,
                     exclusive: false,
                     autoDelete: false);

channel.BasicPublish(exchange: "",
                     routingKey: "durable_queue",
                     basicProperties: props,
                     body: messageBody);
```

CODE EXAMPLE

Let’s see how the errors and the retry logic look like.

QUEUE DOES NOT EXIST

The Producer fails on publishing the message



QUEUE DOES NOT EXIST – Description

When a producer tries to send messages to a queue that doesn't exist in RabbitMQ or a topic that isn't created in Kafka, it will generally result in an error.

Both RabbitMQ and Kafka provide ways to handle such scenarios by:

- Creating the missing destination on the fly
- Pre-configuring error handling
- Applying retry mechanisms.

QUEUE DOES NOT EXIST – How to detect

In **RabbitMQ**, if a producer attempts to publish a message to a queue that doesn't exist, it will throw a 404 "NOT_FOUND" error, indicating the queue could not be found.

RabbitMQ does not automatically create a queue on the producer’s request, so you need to handle this error explicitly to avoid losing messages.

QUEUE DOES NOT EXIST – How to detect

In **Kafka**, if a producer sends a message to a non-existent topic, it may throw an error depending on the broker configuration.

By default, Kafka brokers can be configured to auto-create topics, which can mitigate this error but comes with trade-offs in production environments.

QUEUE DOES NOT EXIST – How to Solve in RabbitMQ

1. Declare the Queue Before Sending

The most common and reliable approach in RabbitMQ is for the producer to declare the queue before publishing the message.

If the queue exists, the QueueDeclare command does nothing
If it doesn't, RabbitMQ creates it. (this is important every time the application startup)

```
var factory = new ConnectionFactory() { HostName = "localhost" };
using (var connection = factory.CreateConnection())
using (var channel = connection.CreateModel())
{
    // Declare the queue (create if not exists)
    channel.QueueDeclare(queue: "task_queue",
                         durable: true,
                         exclusive: false,
                         autoDelete: false,
                         arguments: null);

    // Publish the message
    var messageBody = Encoding.UTF8.GetBytes("Hello, RabbitMQ!");
    channel.BasicPublish(exchange: "",
                         routingKey: "task_queue",
                         basicProperties: null,
                         body: messageBody);
}
```

QUEUE DOES NOT EXIST – How to Solve in RabbitMQ

2. Use Error Handling and Retry Logic

If the queue declaration is not feasible (e.g., the producer doesn't know queue details), wrap the publish operation in a try-catch block and add retry logic.

- **Wait until the queue is created** (useful in systems where queues are dynamically created by other services).
- **Retry a certain number of times**, then log an error or exit if the queue is still missing.

QUEUE DOES NOT EXIST – How to Solve in RabbitMQ

2. Use Error Handling and Retry Logic

```
try
{
    channel.BasicPublish(exchange: "",
                         routingKey: "non_existent_queue",
                         basicProperties: null,
                         body: messageBody);
}
catch (RabbitMQ.Client.Exceptions.OperationInterruptedException ex)
{
    Console.WriteLine("Queue does not exist or is not reachable. Retrying...")
    // Retry or log the error as appropriate
}
```

Use exponential backoff and configure a maximum retry limit.

QUEUE DOES NOT EXIST – How to Solve in RabbitMQ

```
using (var connection = factory.CreateConnection())
using (var channel = connection.CreateModel())
{
    string queueName = "my_queue";
    int retryCount = 0;
    int maxRetries = 5;

    while (retryCount < maxRetries)
    {
```

Retry Logic: If the queue doesn't exist, the consumer retries up to maxRetries (5 in this example) with a 3-second delay between retries. You can adjust the retry count and delay based on your use case.

```
catch (RabbitMQ.Client.Exceptions.OperationInterruptedException ex)
{
    retryCount++;
    Console.WriteLine($"Queue '{queueName}' does not exist. Retry {retryCount}/{maxRetries}
    Thread.Sleep(3000); // Wait for 3 seconds before retrying
}
```

QUEUE DOES NOT EXIST – How to Solve in RabbitMQ

Graceful Exit: If the queue is still not available after the maximum retries, the program logs an error and exits gracefully.

```
if (retryCount == maxRetries)
{
    Console.WriteLine($"Max retries reached. Queue '{queueName}' not found. Exiting...");
}
```

QUEUE DOES NOT EXIST – How to Solve in RabbitMQ

If you prefer **not** to have retries and simply exit the consumer when the queue is not found, you can catch the exception and log the error, then exit the process immediately.

```
catch (RabbitMQ.Client.Exceptions.OperationInterruptedException ex)
{
    // Log and exit
    Console.WriteLine($"Queue '{queueName}' does not exist: {ex.Message}");
    Console.WriteLine("Exiting the consumer...");
    return;
}
```

QUEUE DOES NOT EXIST– How to Solve in RabbitMQ

3. Dead-Letter Exchange or Backup Queue

Configure a dead-letter exchange to route messages to an alternative destination if the queue is missing or has issues.

While this doesn’t solve the missing queue issue directly, it provides a safe fallback for undelivered messages.

QUEUE DOES NOT EXIST – How to Solve in Kafka

In Kafka, **producers cannot directly declare or create topics** when they send messages.

However, Kafka brokers can be configured to **auto-create topics** when a producer tries to publish to a non-existent topic.

QUEUE DOES NOT EXIST – How to Solve in Kafka

1. Enable Auto Topic Creation (Not Recommended for Production)

Kafka brokers can be configured to create topics automatically upon the first request.

In production it lacks control over topic configurations (e.g., partition and replication settings).

• Set auto.create.topics.enable=true in server.properties on the broker.

QUEUE DOES NOT EXIST – How to Solve in Kafka

Best Practice: For production, pre-create topics with appropriate configurations to maintain control over partitions, replication, and topic settings.

There is an option to define the topic and its partitions in the code using Admin Client.

- AdminClient requires elevated permissions
- Giving services admin-level access creates security risks
- Generally, it's not recommended to use AdminClient in production environment

QUEUE DOES NOT EXIST – How to Solve in Kafka

Pre-Create Topics in Production Environments

In production, we manually create topics to ensure configurations align with application needs.

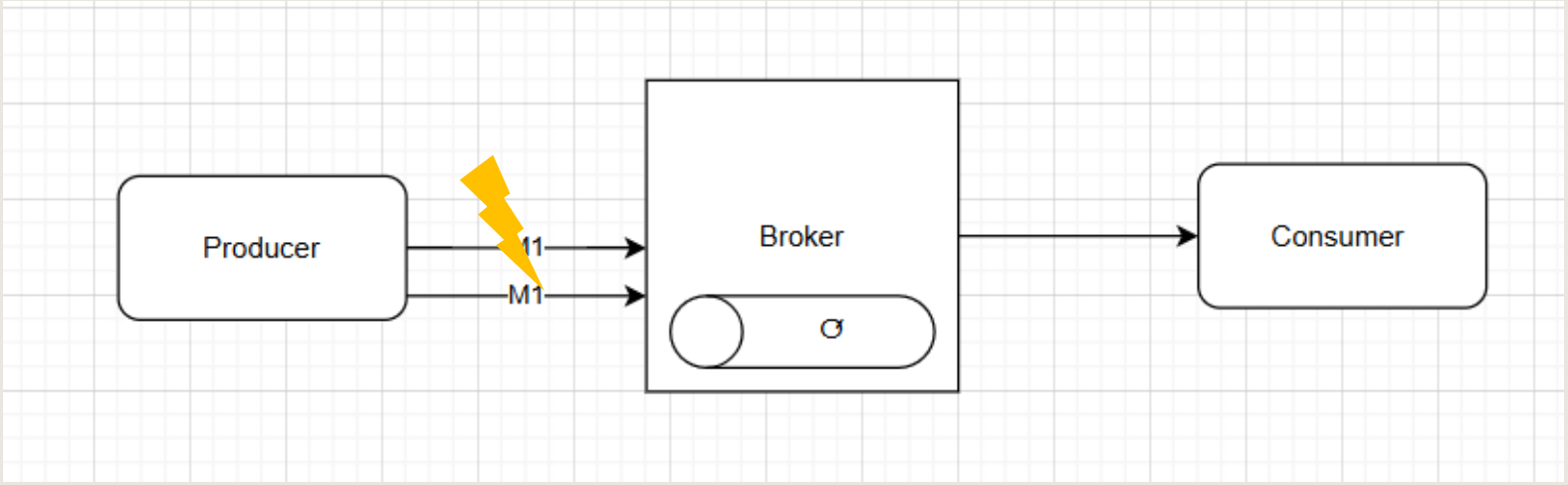
- **Implementation**: Use Kafka CLI or an admin client API to create the topic if it doesn't exist.

Create a separate Admin Service – Separation of responsibilities

```
var adminConfig = new AdminClientConfig { BootstrapServers = "localhost:9092" };
using (var adminClient = new AdminClientBuilder(adminConfig).Build())
{
    try
    {
        await adminClient.CreateTopicsAsync(new List<TopicSpecification>
        {
            new TopicSpecification
            {
                Name = "my_topic",
                NumPartitions = 3,
                ReplicationFactor = 1
            }
        });
    }
    catch (CreateTopicsException e)
    {
        if (e.Results[0].Error.Code != ErrorCode.TopicAlreadyExists)
        {
            Console.WriteLine($"Error creating topic: {e.Results[0].Error.Reason}");
        }
    }
}
```

DUPLICATE MESSAGE ERRORS

The Producer fails on publishing the message



DUPLICATE MESSAGE ERRORS – Description

When implementing retries, there is a risk of **duplicate messages** being sent to RabbitMQ if the producer is uncertain whether the previous attempt succeeded.

Handling Strategy: Use idempotent producer configurations (in Kafka) or implement custom deduplication at the consumer level to ensure message uniqueness.

DUPLICATE MESSAGE ERRORS – How to detect

Producers can’t detect duplicate messages.

Producers can set up relevant information to help detect and prevent duplications

DUPLICATE MESSAGE ERRORS – How to Solve in RabbitMQ

Producers can add a unique message_id in the message properties.

Consumers can then check this ID against a deduplication store to determine if the message has already been processed.

- **Deduplication Store:** Consumers can keep a record of processed message IDs in a deduplication store (e.g., Redis, in-memory cache, or database). When a new message is received, the consumer can check if the ID already exists in the store.

DUPLICATE MESSAGE ERRORS – How to Solve in RabbitMQ

Implementation:

- Store each message ID in Redis (or similar) with a time-to-live (TTL) to prevent unlimited growth.
- Check the store before processing each message.

RABBITMQ SOLUTION

DUPLICATE MESSAGE ERRORS – How to Solve in RabbitMQ

Publisher Confirms provide a reliable way for producers to know whether a message has been successfully handled by the broker.

When publisher confirms are enabled, RabbitMQ sends an acknowledgment (ACK) to the producer once the message has been delivered to the queue. If the broker is unable to deliver the message, a negative acknowledgment (NACK) is sent.

DUPLICATE MESSAGE ERRORS – How to Solve in RabbitMQ

• **Enable Publisher Confirms**: Use the ConfirmSelect() method to enable publisher confirms.

• **Wait for Acknowledgments**: After publishing, use the WaitForConfirmsOrDie() method to wait for the broker's acknowledgment.

• **Retry on NACK**: If the broker sends a NACK, the producer can retry sending the message.

DUPLICATE MESSAGE ERRORS – How to Solve in RabbitMQ

Best Practice:

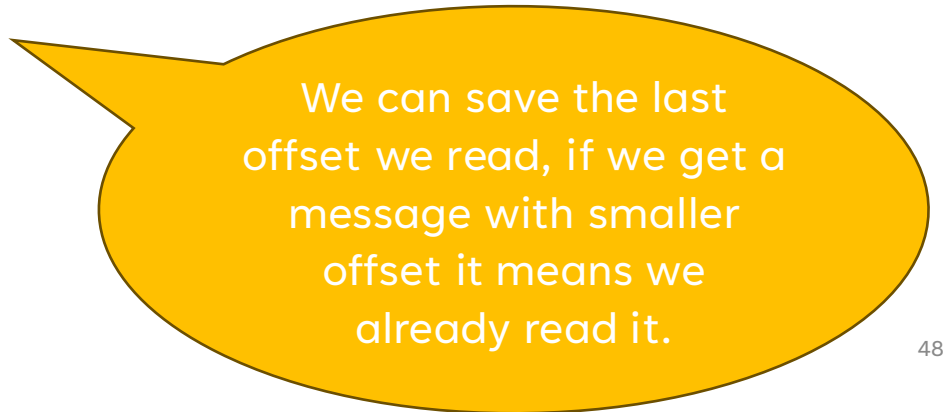
- **Always enable publisher confirms** for critical message delivery to ensure that the message is safely received by the broker.
- Implement **retry logic** when a NACK is received to avoid message loss.

```
channel.ConfirmSelect();

channel.BasicPublish(exchange: "logs",
                     routingKey: "info",
                     basicProperties: props,
                     body: messageBody);

try
{
    channel.WaitForConfirmsOrDie();
}
catch (Exception ex)
{
    // Handle failure (e.g., retry publishing)
    Console.WriteLine("Message failed to be confirmed: " + ex.Message);
}
```

DUPLICATE MESSAGE ERRORS – How to Solve in Kafka

Offset Tracking: Kafka consumers process messages based on offsets, which ensures messages are processed in order within a partition.

Each message in Kafka has a unique offset within a partition, making it easy to identify duplicates at the consumer level.



DUPLICATE MESSAGE ERRORS – How to Solve in Kafka

Kafka provides an **idempotent producer** mode.
Enabling idempotence ensures that duplicate messages are avoided at the broker level.

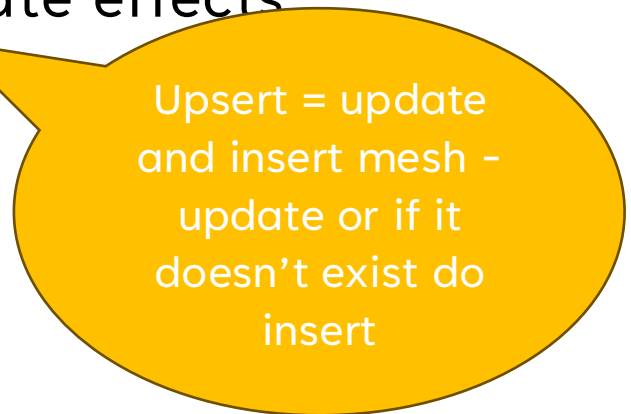
The Kafka producer assigns a unique sequence number to each message within a session, and the broker only stores a message once for each unique sequence.

```
var config = new ProducerConfig
{
    BootstrapServers = "localhost:9092",
    EnableIdempotence = true
};
```

DUPLICATE MESSAGE ERRORS – How to Solve in Kafka

Idempotent or Deduplication Logic in Consumer:

- For situations where idempotence cannot be fully enforced, design consumer logic to handle duplicates. Similar to RabbitMQ, a deduplication store (e.g., Redis or database) can be used.
- **Best Practice**: Use upserts or conditional writes in downstream processing to avoid duplicate effects.



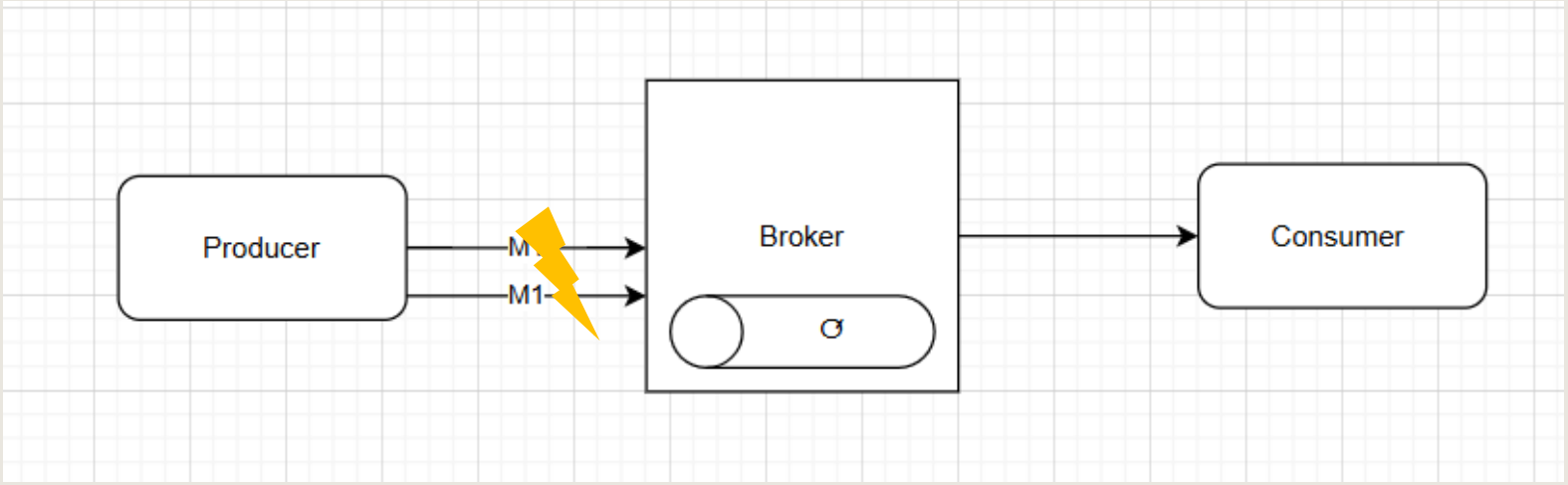
DUPLICATE MESSAGE ERRORS – SUMMARY

Feature	RabbitMQ	Kafka
Automatic Deduplication	Not built-in; use message ID and deduplication store	Idempotent producer (automatic deduplication at broker)
Idempotent Producer	Not available	`enable.idempotence=true` for deduplication at producer
Deduplication Store	External store recommended (e.g., Redis, cache)	Not needed if EOS or idempotence enabled; fallback if not
Best Practice	Use idempotent operations, deduplication store	Use idempotent producer or EOS for stronger guarantees

EOS – Exactly Once Strategy

SERIALIZATION/DESERIALIZATION ERRORS

The message does not contain the expected schema



SERIALIZATION/DESERIALIZATION ERRORS – Description

Serialization and deserialization errors are common in messaging systems like Kafka and RabbitMQ, especially when producers and consumers use incompatible data formats or schema changes aren’t managed correctly.

Handling these errors effectively is essential for reliable data processing.

SERIALIZATION/DESERIALIZATION ERRORS – How to detect

Serialization issues may occur either during the message production (serialization) or message consumption (deserialization) stages.

Here’s how to detect these issues:

- **Error Logs**: When the producer fails to serialize a message, a SerializationException is usually thrown in Kafka or JsonException, XmlException in RabbitMQ, or any other format-specific exception.

This error should be logged, often containing details about the specific field or type that failed to serialize.

SERIALIZATION/DESERIALIZATION ERRORS – How to detect

• **Monitoring**: In Kafka we can set up monitoring or logging alerts on producer error logs.

You can configure alerts to detect SerializationException or any failed produce requests due to serialization issues.

SERIALIZATION/DESERIALIZATION ERRORS – RabbitMQ issue description

In RabbitMQ, **serialization** happens when the producer encodes data into a specific format (e.g., JSON, XML),

Deserialization occurs on the consumer side.

Since RabbitMQ doesn’t natively handle schema management, it’s up to the application to enforce consistency.

SERIALIZATION/DESERIALIZATION ERRORS – RabbitMQ issue description

Common Causes of Ser/Deser Errors in RabbitMQ

• **Schema Changes**: Similar to Kafka, schema evolution can lead to incompatible formats if the consumer is not updated to handle new data structures.

• **Inconsistent Message Properties**: RabbitMQ allows setting message headers like content_type (e.g., application/json), which the consumer may rely on to select a deserializer. Misconfigured or inconsistent headers lead to deserialization failures.

• **Mismatched Serialization Logic**: If producers and consumers are using different serializers, messages may not be correctly deserialized (e.g., producer uses JSON but consumer expects XML).

SERIALIZATION/DESERIALIZATION ERRORS – How to Solve in RabbitMQ

1. Explicit Content-Type Header:
Use the content_type header to inform consumers about the message format (e.g., application/json, application/xml).
Consumers can then select the appropriate deserializer.

• **Best Practice**: Standardize content types and ensure all consumers recognize these standards. This is especially useful for applications that support multiple formats.

SERIALIZATION/DESERIALIZATION ERRORS – How to Solve in RabbitMQ

2. Schema Enforcement with External Validation:
Implement schema validation libraries on the consumer side to validate messages against an expected schema before deserializing. Libraries such as JSON Schema Validator can ensure compatibility.

- **Best Practice**: Use a shared schema repository for validation and version control. This approach works well in environments where a Schema Registry isn't available.

SERIALIZATION/DESERIALIZATION ERRORS – How to Solve in RabbitMQ

3. Dead Letter Exchange (DLX):
Configure a Dead Letter Exchange to route invalid messages to a separate queue if deserialization fails. This enables consumers to continue processing valid messages.

- **Best Practice**: Configure the DLX and monitor it to catch and investigate deserialization issues as they occur. This can be set at the queue level in RabbitMQ.

SERIALIZATION/DESERIALIZATION ERRORS – How to Solve in RabbitMQ

4. Idempotent Consumer Logic:
Ensure that consumer logic can handle reprocessing of messages if retries or restarts are required after deserialization failures.

- **Best Practice**: Design idempotent processing workflows so that duplicate or reprocessed messages do not impact system state.

SERIALIZATION/DESERIALIZATION ERRORS – How to Solve in RabbitMQ

RabbitMQ Consumer Example with Deserialization Error Handling in .NET

```
public void OnMessageReceived(BasicDeliverEventArgs ea)
{
    try
    {
        var body = ea.Body.ToArray();
        var message = JsonSerializer.Deserialize<MyMessage>(body);
        // Process the message here
    }
    catch (JsonException ex)
    {
        Console.WriteLine($"Deserialization error: {ex.Message}");
        // Send to Dead Letter Exchange
        SendToDLX(ea);
    }
}

private void SendToDLX(BasicDeliverEventArgs ea)
{
    var dlxChannel = connection.CreateModel();
    dlxChannel.BasicPublish(exchange: "my_dlx_exchange", routingKey: ea.RoutingKey,basicProperties: ea.BasicProperties, body: ea.Body);
}
```

SERIALIZATION/DESERIALIZATION ERRORS – Kafka issue description

In Kafka, **serialization** occurs when a producer encodes data into a specific format (e.g., JSON, Avro) before sending it to a topic, while **deserialization** happens when a consumer decodes the data.

Since Kafka is schema-agnostic, it does not enforce a specific format, leaving this responsibility to the producer and consumer code.

SERIALIZATION/DESERIALIZATION ERRORS – Kafka issue description

Common Causes of Ser/Deser Errors in Kafka

- **Schema Evolution**: Schema changes, such as adding, removing, or modifying fields, can cause incompatibility if consumers expect a different schema than what the producer uses.
- **Mismatched Ser/Deser Configuration**: Producers and consumers must use the same serialization format (e.g., JSON, Avro). A mismatch (e.g., producer uses JSON, consumer expects Avro) leads to deserialization errors.
- **Null or Empty Messages**: Kafka does not enforce a specific schema or data validation, so messages without expected fields can lead to runtime errors on the consumer side.

SERIALIZATION/DESERIALIZATION ERRORS – How to Solve in Kafka

1. Schema Management with Schema Registry:

Use a Schema Registry (e.g., Confluent Schema Registry) to store and manage schemas for Kafka topics.

Schema Registry enforces schema compatibility rules (backward, forward, or full compatibility) between producer and consumer applications.

```
var config = new ProducerConfig { BootstrapServers = "localhost:9092" };
using var producer = new ProducerBuilder<string, MyRecord>(config)
    .SetKeySerializer(new AvroSerializer<string>(schemaRegistry))
    .SetValueSerializer(new AvroSerializer<MyRecord>(schemaRegistry))
    .Build();
```

SERIALIZATION/DESERIALIZATION ERRORS – How to Solve in Kafka

2. Error Handling with Dead Letter Queue (DLQ):
Route problematic messages to a Dead Letter Queue (DLQ) when deserialization fails, allowing consumers to continue processing valid messages.

- **Best Practice**: Use a separate topic as the DLQ and set up monitoring to investigate errors. This can be achieved in Kafka Streams or Kafka Connect, which supports DLQs natively for serialization errors.

SERIALIZATION/DESERIALIZATION ERRORS – How to Solve in Kafka

3. Retry Logic with Circuit Breaker Pattern:
Implement retry logic with a circuit breaker pattern, especially if transient schema or network issues are expected. If deserialization fails, retry a limited number of times before sending the message to DLQ.

- **Best Practice**: Limit retries to avoid endless loops and use exponential backoff to minimize impact on system resources.

SERIALIZATION/DESERIALIZATION ERRORS – How to Solve in Kafka

4. Use Fallback Deserialization:
Set up a fallback deserialization handler for cases where the schema version is unknown. This allows the consumer to ignore fields it doesn’t recognize.

•**Best Practice**: Implement custom deserializers with exception handling to manage

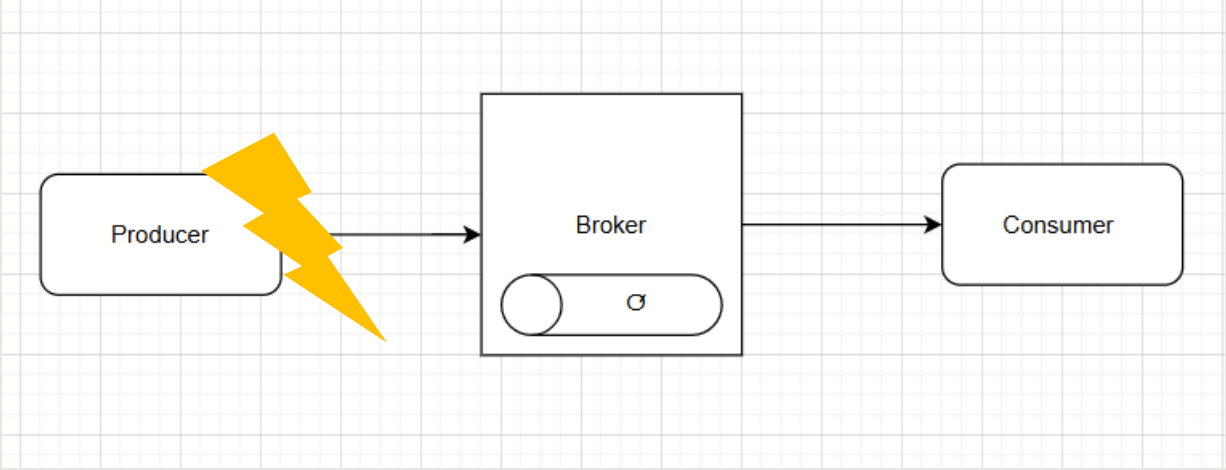
SERIALIZATION/DESERIALIZATION ERRORS – How to Solve in Kafka

Kafka Consumer Example with Deserialization Error Handling

```
try
{
    var consumeResult = consumer.Consume();
    var message = consumeResult.Message.Value;
    // Process message here
}
catch (ConsumeException e) when (e.Error.Code == ErrorCode.Local_ValueDeserialization)
{
    // Log error and send message to DLQ
    Console.WriteLine($"Deserialization error: {e.Error.Reason}");
    dlqProducer.Produce("my_dlq_topic", new Message<Null, string> { Value = e.Message.Value });
}
```

AUTHORIZATION AND AUTHENTICATION ERRORS

Errors occur if the producer is not authorized to write to the broker or if authentication fails.



AUTHORIZATION AND AUTHENTICATION ERRORS – Description

Authorization and authentication errors in RabbitMQ and Kafka are common in systems that require secure access to queues, topics, and data streams.

These errors generally arise when a user or application does not have the correct credentials or permissions to access certain resources.

AUTHORIZATION AND AUTHENTICATION ERRORS – Description

Types of Errors

1.Authentication Errors:

- **Error**: Occurs when the producer or consumer does not provide valid credentials.

- **Common Causes**: Missing or incorrect credentials, misconfigured security protocol, expired certificates, or unsupported authentication mechanisms.

AUTHORIZATION AND AUTHENTICATION ERRORS – Description

2.Authorization Errors:

- **Error**: Occurs when a user does not have the required permissions to perform an action (e.g., produce to a topic, actions on exchanges).

- **Common Causes**: Misconfigured ACLs (Access Control Lists) or lack of topic-level permissions.

AUTHORIZATION AND AUTHENTICATION ERRORS – How to detect

• **Error Messages**: Kafka and Rabbit logs detailed error messages for authentication and authorization errors.

• **Logs**: Check client logs (producer or consumer) for specific messages like:
•Authentication failed or AuthorizationException.

• **Monitoring Tools**: Use monitoring tools like Confluent Control Center, RabbitMQ Management Plugin, Prometheus, or Grafana with metrics to track authorization and authentication failures.

AUTHORIZATION AND AUTHENTICATION ERRORS – How to Solve in RabbitMQ

1. **Authentication Configuration**:

- **Fix**: Ensure the client application is using valid credentials (username and password) configured in RabbitMQ. Update these in your application configuration if needed.

```
var factory = new ConnectionFactory()
{
    HostName = "localhost",
    UserName = "guest",
    Password = "guest"
};
```

In real applications we will **NOT** put the credentials exposed in the code! Must use secret manager!

- **Best Practice**: Avoid using default credentials (guest/guest) in production. Create unique users with strong passwords and rotate them periodically.

AUTHORIZATION AND AUTHENTICATION ERRORS – How to Solve in RabbitMQ

2. Authorization Configuration:

- **Fix**: Set up permissions on RabbitMQ users to access only specific resources.
- Permissions are applied to users on specific virtual hosts, limiting access to exchanges, queues, and routing keys.

- **Best Practice**: Apply the principle of least privilege and avoid giving users broad permissions. Limit users to specific actions (e.g., read, write) and specific queues/exchanges.

AUTHORIZATION AND AUTHENTICATION ERRORS – How to Solve in RabbitMQ

3. TLS/SSL Configuration for Enhanced Security:

TLS – Transport Layer Security

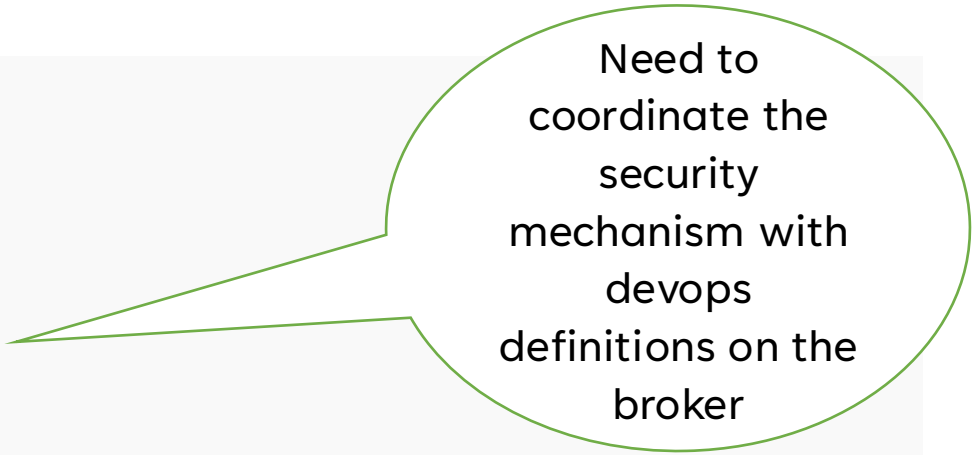
SSL – Secure Sockets Layer

- **Fix**: If using TLS, ensure that SSL certificates are configured correctly and that the RabbitMQ server and client can negotiate a connection.

AUTHORIZATION AND AUTHENTICATION ERRORS – How to Solve in RabbitMQ

- **Best Practice**: Regularly update certificates before expiration and review SSL policies on RabbitMQ to enforce strong encryption.

```
var factory = new ConnectionFactory
{
    HostName = "localhost",
    Ssl = new SslOption
    {
        Enabled = true,
        ServerName = "my_server",
        CertificateValidationCallback = (sender, certificate, chain, sslPolicyErrors) => true
    }
};
```



AUTHORIZATION AND AUTHENTICATION ERRORS – How to Solve in Kafka

1. Authentication Configuration:

- **Fix**: Ensure that the producer and consumer have valid credentials. Update the Kafka client configuration with the correct credentials, security protocol, and SASL mechanism.

```
var config = new ProducerConfig
{
    BootstrapServers = "localhost:9092",
    SaslMechanism = SaslMechanism.Plain,
    SecurityProtocol = SecurityProtocol.SaslPlaintext,
    SaslUsername = "user",
    SaslPassword = "password"
};
```

Need to coordinate the security mechanism with devops definitions on the broker

AUTHORIZATION AND AUTHENTICATION ERRORS – How to Solve in Kafka

2. Authorization Configuration:

• **Fix**: Update ACLs to grant necessary permissions for the client's user. ACLs are managed at the broker level and need to allow specific actions for each client.

• **Best Practice**: Use least-privilege principles, giving clients only the permissions they need to operate. Regularly audit ACLs to avoid accidental privilege escalation.

AUTHORIZATION AND AUTHENTICATION ERRORS – How to Solve in Kafka

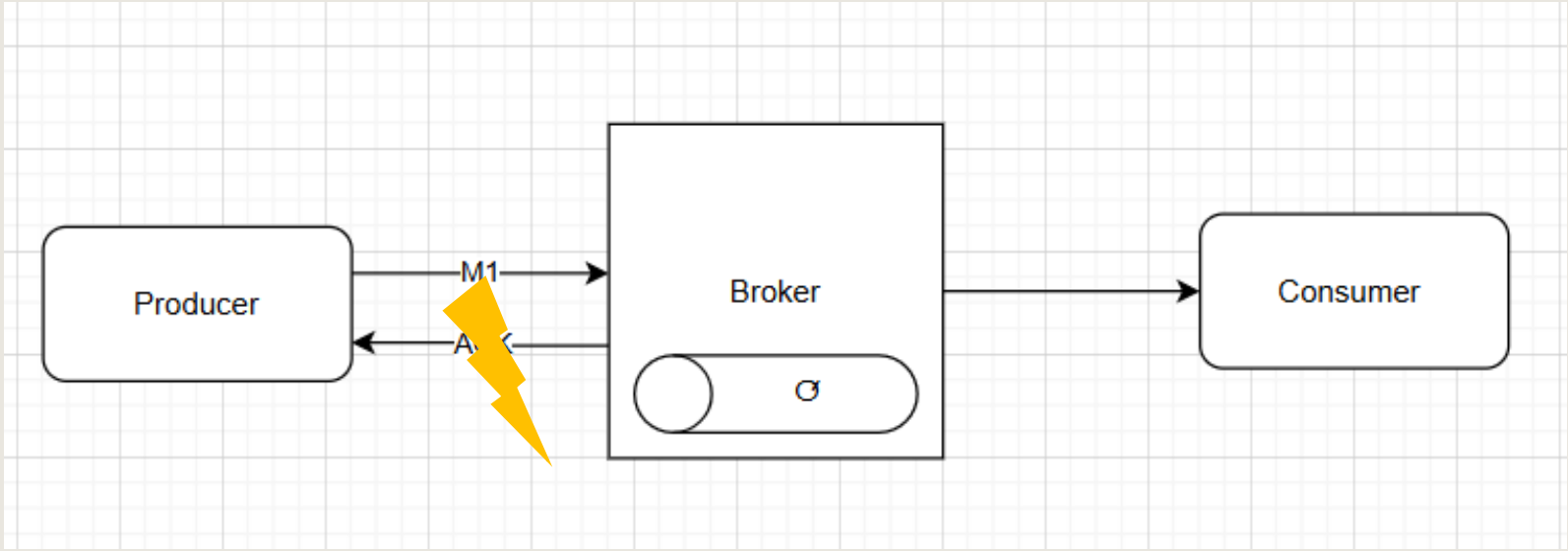
3. Certificate and Protocol Configuration:

• **Fix**: For TLS/SSL configurations, ensure certificates are valid, not expired, and that client and broker support the same SSL protocol versions.

• **Best Practice**: Regularly update certificates before expiration and periodically test all Kafka client configurations, especially after upgrades or certificate rotations.

TIMEOUT ERRORS

If the broker does not acknowledge the message within a specified timeout, it can trigger an error on the producer.



TIMEOUT ERRORS – Description

- Timeout errors in RabbitMQ and Kafka can disrupt message flow between producers and consumers.
- Generally caused by network latency, resource constraints, or configuration issues.

TIMEOUT ERRORS – Description

Types of Timeout Errors

1. Connection Timeout:

- **Error**: Connection to broker times out.
- **Cause**: Network issues, firewall rules, or overloaded broker.
- **Detection**: Check broker logs and application logs for connection timeout errors.

TIMEOUT ERRORS – Description

2. **Request Timeout** :
   - **Error**: Occurs when a request (such as producing a message) fails to complete within the expected time.

   - **Cause**: High network latency, broker overload, or issues with the broker.

   - **Detection**: Look for TimeoutException messages in the client logs.

TIMEOUT ERRORS – Description

3. **Delivery Timeout** :

   •**Error**: Indicates that a message could not be delivered within the delivery.timeout.ms period.

   • **Cause**: Network issues, broker overload, or topic congestion.

   • **Detection**: Logs will show DeliveryTimeoutException errors, indicating delivery issues.

TIMEOUT ERRORS – Description

4. Consumer Timeout (TimeoutException):

- **Error**: Happens when a consumer poll operation times out while waiting for messages.

- **Cause**: Slow network or consumer lag (if the consumer can't keep up with message processing).

- **Detection**: TimeoutException in consumer logs indicates poll timeouts.

TIMEOUT ERRORS – How to Solve in RabbitMQ

1. Configure Client-Side Timeouts:

• **Connection Timeout**: Set ConnectionFactory timeout settings to handle network latency more gracefully.

```
var factory = new ConnectionFactory
{
    HostName = "localhost",
    RequestedConnectionTimeout = 30000 // 30 seconds
};
```

TIMEOUT ERRORS – How to Solve in RabbitMQ

2. Broker Resource Allocation: (DEVOPS responsibility)

• Ensure the RabbitMQ server has adequate memory and CPU to prevent overloading, which can lead to timeouts.

• Monitor RabbitMQ resource metrics using tools like RabbitMQ Management Plugin or Prometheus for proactive troubleshooting.

TIMEOUT ERRORS – How to Solve in Kafka

1. **Increase Timeout Configurations**:
   Producer Configuration:
   Adjust request.timeout.ms and delivery.timeout.ms for longer wait periods if network latency is expected.

```
var config = new ProducerConfig
{
    BootstrapServers = "localhost:9092",
    RequestTimeoutMs = 30000,    // 30 seconds
    DeliveryTimeoutMs = 60000    // 60 seconds
};
```

TIMEOUT ERRORS – How to Solve in Kafka

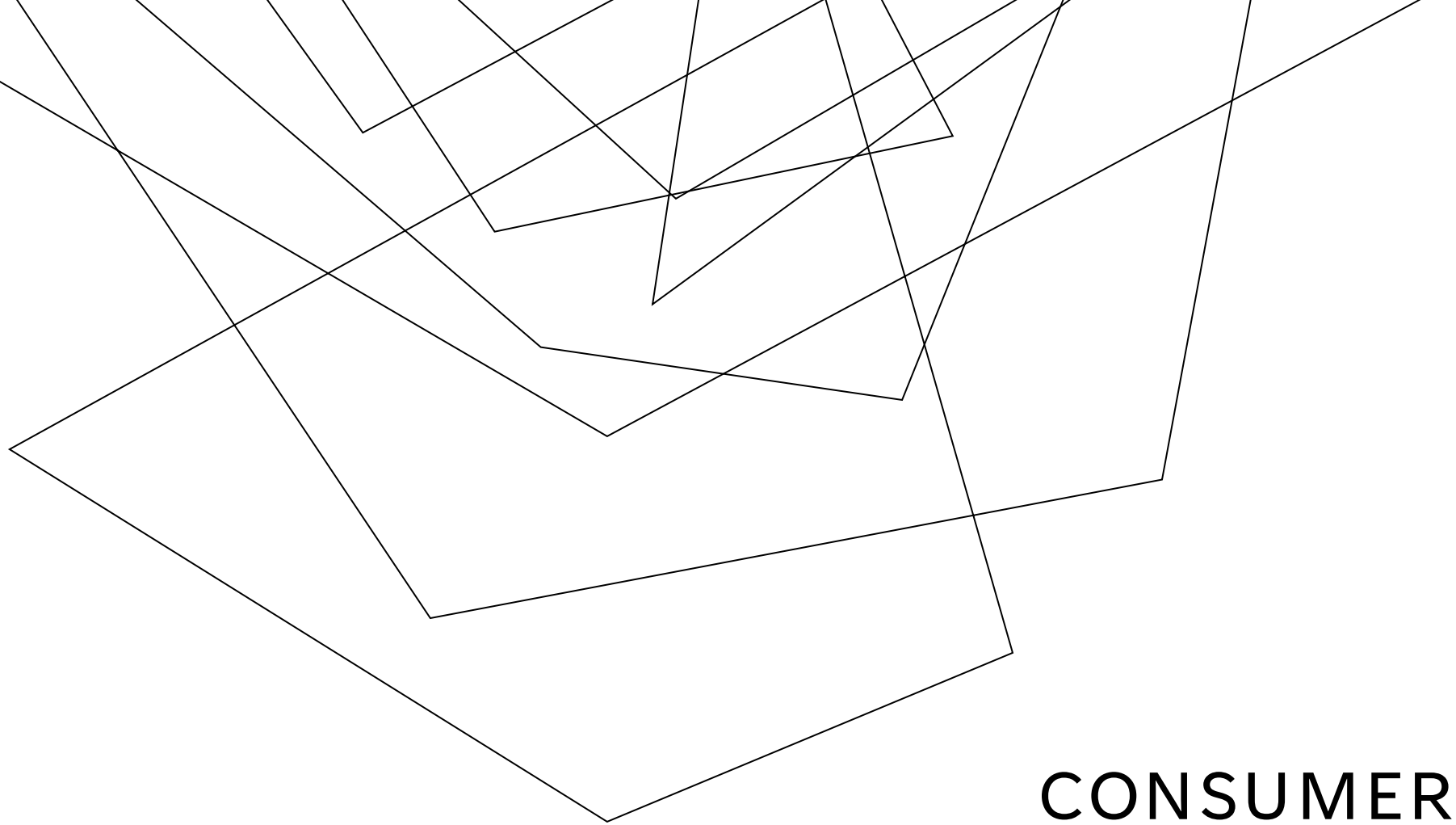
Consumer Configuration:

Adjust fetch.max.wait.ms and session.timeout.ms to allow more time for data retrieval and rebalancing.

TIMEOUT ERRORS – How to Solve in Kafka

2. Batch Size and Retries:
Configure retries and acks settings to optimize batch sizes and retry counts, which can reduce the load on brokers and improve delivery rates under high loads.

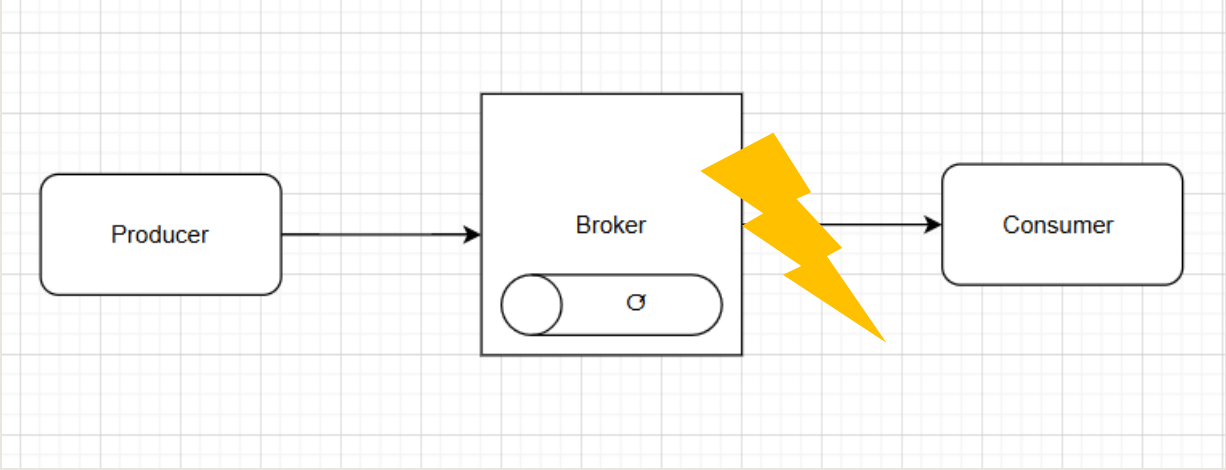
3. Monitoring and Alerting:
Set up alerts for timeout errors using monitoring tools to detect latency spikes or service disruptions promptly.



CONSUMER-SIDE ERROR HANDLING

NETWORK ERRORS

Network errors on the consumer side are typically associated with disconnections, reconnection failures, or latency-related issues.



NETWORK ERRORS – Description

Connection Loss:

• **Error**: Consumers lose connection to the server/broker due to network issues, resulting in abrupt disconnections.

• **Cause**: Network instability, firewall rules, or resource limitations on the broker.

• **Detection**: Error messages like Connection closed unexpectedly, appear in consumer logs.

NETWORK ERRORS – How to Solve in RabbitMQ

1. **Retry and Reconnect Logic**:

- Implement retry logic in the consumer code to handle network disruptions gracefully.

NETWORK ERRORS – How to Solve in RabbitMQ

2. Use Auto-Recovery:

- Enable **Automatic Recovery** in RabbitMQ clients, which automatically re-establishes the connection and re-subscribes to the queue.

```
var factory = new ConnectionFactory
{
    AutomaticRecoveryEnabled = true
};
```

NETWORK ERRORS – How to Solve in RabbitMQ

3. Network and Resource Monitoring:

- Use RabbitMQ’s Management Plugin or monitoring tools (like Prometheus and Grafana) to track network latency, connection stability, and resource usage.

NETWORK ERRORS – Kafka specific errors

Consumer Rebalance Timeout:

• **Error**: TimeoutException or RebalanceInProgressException due to delays during group rebalancing.

• **Cause**: Network delays prevent the consumer from joining the group or completing a rebalance within the configured timeout.

• **Detection**: Logs show RebalanceInProgressException or TimeoutException with rebalancing-specific messages.

NETWORK ERRORS – Kafka specific errors

Fetch Timeout:

• **Error**: The consumer fails to fetch messages within the configured fetch.max.wait.ms period.

• **Cause**: Network issues or broker delays.

• **Detection**: Errors like TimeoutException in logs indicate that fetch requests timed out.

NETWORK ERRORS – How to Solve in Kafka

1. Configure Timeout Settings:

- Adjust consumer timeouts to account for network latency.

```
var config = new ConsumerConfig
{
    BootstrapServers = "localhost:9092",
    SessionTimeoutMs = 45000,      // Adjust as needed
    HeartbeatIntervalMs = 15000,
    FetchMaxWaitMs = 500           // for delayed message fetching
};
```

NETWORK ERRORS – How to Solve in Kafka

2. Backoff and Retry Logic:

- Implement exponential backoff on retry attempts to avoid overloading the network or broker.
- Use retry policies in the consumer application to handle network-related failures gracefully.

NETWORK ERRORS – How to Solve in Kafka

3. Handle Rebalances Efficiently:

Reminder – rebalance occur when consumers are added to the consumer group

- If rebalancing is causing timeouts, consider adjusting max.poll.interval.ms and session.timeout.ms to prevent consumers from being removed from the group during high-latency periods.

- For long-running processes, call poll() periodically to avoid rebalance-triggering timeouts.

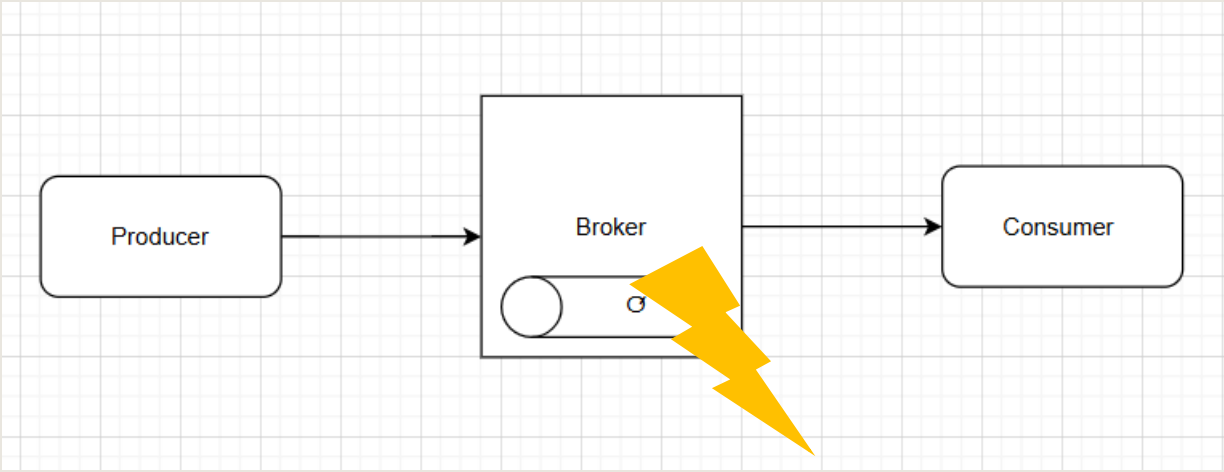
NETWORK ERRORS – How to Solve in Kafka

4. Enable Monitoring:

- Use monitoring tools like Confluent Control Center or Grafana to keep track of consumer lag, network latency, and other health metrics. Set up alerts for metrics that can indicate network issues, such as high consumer lag or missed heartbeats.

QUEUE DOES NOT EXIST

The Consumer fails on consuming the message



QUEUE DOES NOT EXIST – Description

In RabbitMQ, queues are entities created on the broker that consumers subscribe to for receiving messages. If a consumer tries to consume from a queue that doesn’t exist, it encounters a channel or consumer error.

If a topic does not exist, Kafka may behave differently depending on the cluster configuration.

QUEUE DOES NOT EXIST – How to Solve in RabbitMQ

Check if the Queue Exists Before Consuming

- You can use QueueDeclarePassive to check if a queue exists. This method only checks the existence of the queue without modifying or declaring it. If the queue doesn't exist, it throws an exception, which you can catch to handle the error.

```
var response = channel.QueueDeclarePassive("queue-name");
// returns the number of messages in Ready state in the qu
response.MessageCount;
// returns the number of consumers the queue has
response.ConsumerCount;
```

If the entity does not exist, the operation fails with a channel level exception.

QUEUE DOES NOT EXIST – How to Solve in RabbitMQ

- **QueueDeclarePassive**: This method checks if the queue exists. It throws an exception if the queue does not exist without declaring a new queue.

- **Exception Handling**: If the queue doesn't exist, a RabbitMQ.Client.Exceptions.OperationInterruptedException will be thrown, which you can catch and handle (e.g., log the error or notify someone).

```
try
{
    // Try to declare the queue passively, which checks for existence only
    channel.QueueDeclarePassive(queueName);
    Console.WriteLine($"Queue '{queueName}' exists. Starting consumption...");

    var consumer = new EventingBasicConsumer(channel);
    consumer.Received += (model, ea) =>
    {
        var body = ea.Body.ToArray();
        var message = System.Text.Encoding.UTF8.GetString(body);
        Console.WriteLine($"[Consumer] Received '{message}'");

        // Acknowledge the message
        channel.BasicAck(ea.DeliveryTag, false);
    };

    channel.BasicConsume(queue: queueName, autoAck: false, consumer: consumer)
}
catch (RabbitMQ.Client.Exceptions.OperationInterruptedException ex)
{
    // Handle the case where the queue does not exist
    Console.WriteLine($"Queue '{queueName}' does not exist: {ex.Message}");
}
```

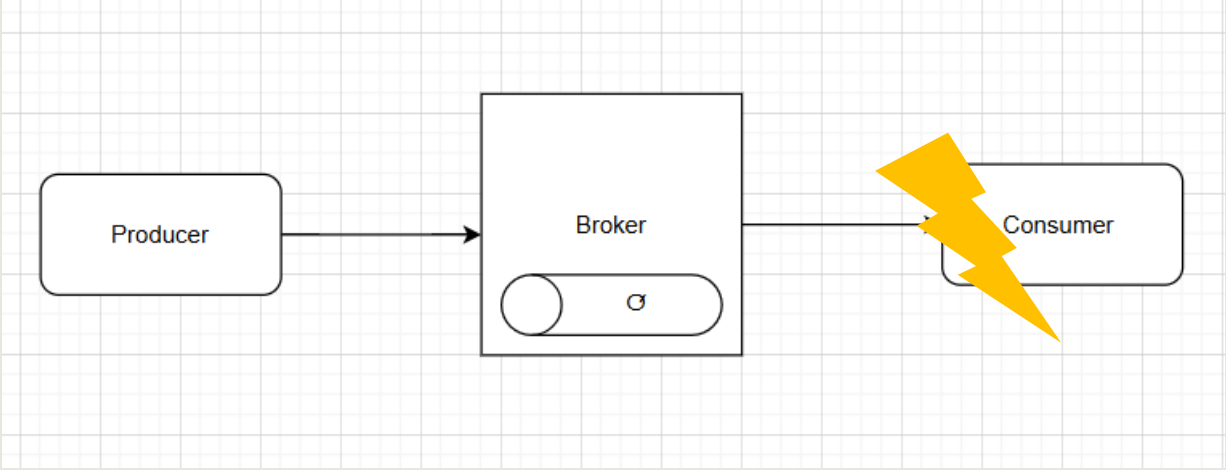
QUEUE DOES NOT EXIST– How to Solve in Kafka

We handle this error in Kafka is the same as the producer:

1. Automatic Topic Creation (if enabled)
2. Admin Client for Topic Creation
3. Check Topic Existence with Retry
4. Log and Monitor

DESERIALIZATION ERRORS

Errors occur if the message format does not match the consumer's expected schema.



DESERIALIZATION ERRORS – Description

- Deserialization errors on the consumer side in RabbitMQ and Kafka happen when the message format or data structure cannot be correctly decoded into the expected format.

- These errors are common in systems where multiple producers and consumers interact with messages, especially when the data schema or serialization format changes without adequate version management.

DESERIALIZATION ERRORS – DETECTION

Consumer logs will indicate serialization errors with specific messages

DESERIALIZATION ERRORS – How to Solve in RabbitMQ

1. Schema Versioning:

Implement versioning for message payloads, especially if the message schema changes over time.

Include a version field in the message body, allowing consumers to deserialize based on the version they support.

DESERIALIZATION ERRORS – How to Solve in RabbitMQ

Define Different Message Classes for Each Version:

```
public class MessageV1
{
    public string Property1 { get; set; }
}

public class MessageV2
{
    public string Property1 { get; set; }
    public string NewProperty { get; set; }
}
```

DESERIALIZATION ERRORS – How to Solve in RabbitMQ

In the publisher side, add the version to the message header

```
// Add version information to message headers
properties.Headers = new Dictionary<string, object>
{
    { "version", version }
};
```

```
channel.BasicPublish(
    exchange: "my_exchange",
    routingKey: "my_key",
    basicProperties: properties,
    body: body
);
```

DESERIALIZATION ERRORS – How to Solve in RabbitMQ

The consumer will extract the header like:

```
public static object DeserializeMessage(string messageBody)
{
    var document = JsonDocument.Parse(messageBody);

    if (document.RootElement.TryGetProperty("version", out var versionElement))
    {
        int version = versionElement.GetInt32();

        // Deserialize based on schema version
        return version switch
        {
            1 => JsonSerializer.Deserialize<MessageV1>(messageBody),
            2 => JsonSerializer.Deserialize<MessageV2>(messageBody),
            _ => throw new InvalidOperationException("Unsupported schema version")
        };
    }
    else
    {
        throw new InvalidOperationException("Schema version not specified");
    }
}
```

DESERIALIZATION ERRORS – How to Solve in RabbitMQ

2. Consumer Validation and Error Handling:

- Use validation to catch deserialization issues and handle them gracefully.
- For JSON or XML, wrap deserialization in a try-catch block and log or handle errors as needed.

```
try
{
    var message = JsonSerializer.Deserialize<MyMessage>(messageBody);
}
catch (JsonException ex)
{
    // Handle or log the deserialization error
}
```

DESERIALIZATION ERRORS – How to Solve in RabbitMQ

3. Fallback or Default Values:

- Implement default values or fallback deserialization mechanisms to handle missing fields or new unexpected fields.
- Libraries like Newtonsoft.Json can be configured to ignore unknown properties, which helps when schema evolution is minimal.

DESERIALIZATION ERRORS – How to Solve in Kafka

1. **Schema Registry (for Avro/Protobuf)**:

- Use Confluent Schema Registry to enforce schema versioning and compatibility (backward, forward, or full).
- Consumers can fetch the correct schema version for each message, avoiding mismatches.

DESERIALIZATION ERRORS – How to Solve in Kafka

2. Handle Unknown Fields:

- Configure the deserialization library to ignore unknown fields if backward compatibility is necessary. Avro, for instance, allows optional fields and default values, making backward and forward compatibility easier.

DESERIALIZATION ERRORS – How to Solve in Kafka

3. Configure Dead-Letter Queue (DLQ):

- Set up a DLQ for messages that fail deserialization. Instead of discarding or blocking the consumer, messages with incompatible schemas are sent to a DLQ, allowing for later analysis or processing.

```
var builder = new ConsumerBuilder<string, MyMessage>(config)
    .SetErrorHandler((_, e) => {
        if (e.Code == ErrorCode.SerializationError)
        {
            // Send message to DLQ or log it for analysis
        }
    });
```

DESERIALIZATION ERRORS – How to Solve in Kafka

4. Implement Deserialization Retry Logic:

• Retry deserialization for transient errors (e.g., network issues with Schema Registry) and log persistent errors for investigation.

• Use exponential backoff or other retry strategies to minimize load.

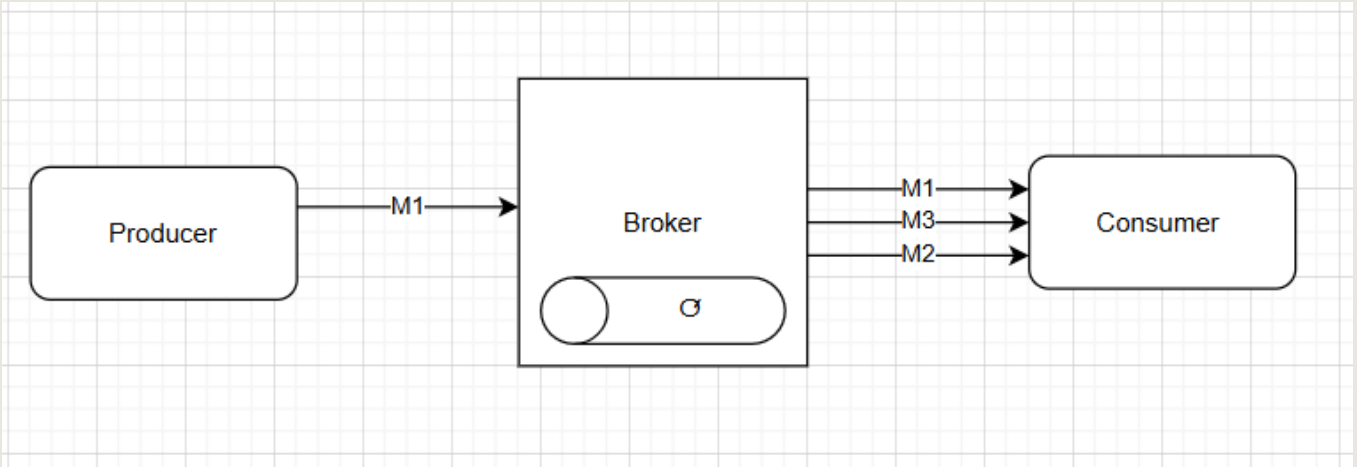
DESERIALIZATION ERRORS – How to Solve in Kafka

5. Log Deserialization Errors with Schema Details:

• Capture specific schema details or field names causing errors in logs for efficient debugging. Tools like Kafka Connect and Confluent Control Center can help track these errors at scale.

MESSAGE ORDER ERRORS

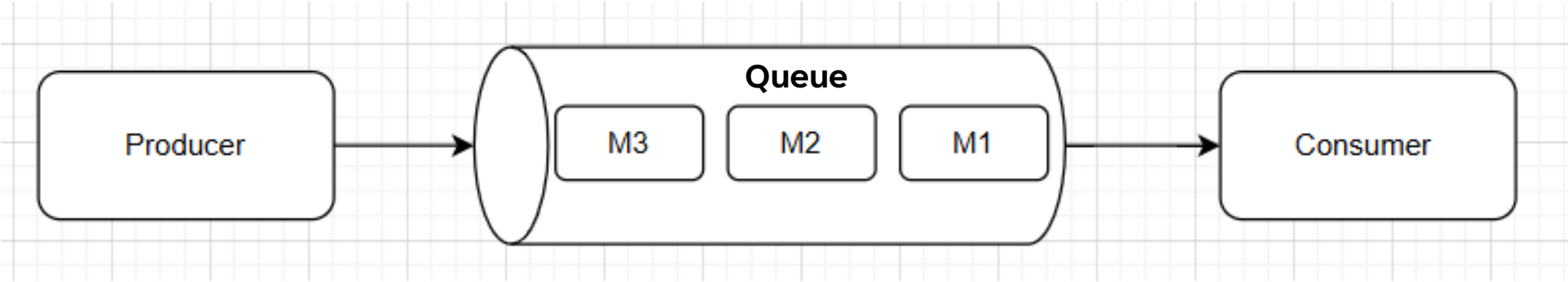
If consumers expect strict message ordering (e.g., with Kafka's partition ordering) but receive messages out of order, it can cause processing issues.



MESSAGE ORDER ERRORS – Description

Kafka and RabbitMQ don’t act the same way regarding ordering.

- Kafka guarantees order only within a partition
- RabbitMQ doesn't inherently guarantee message ordering

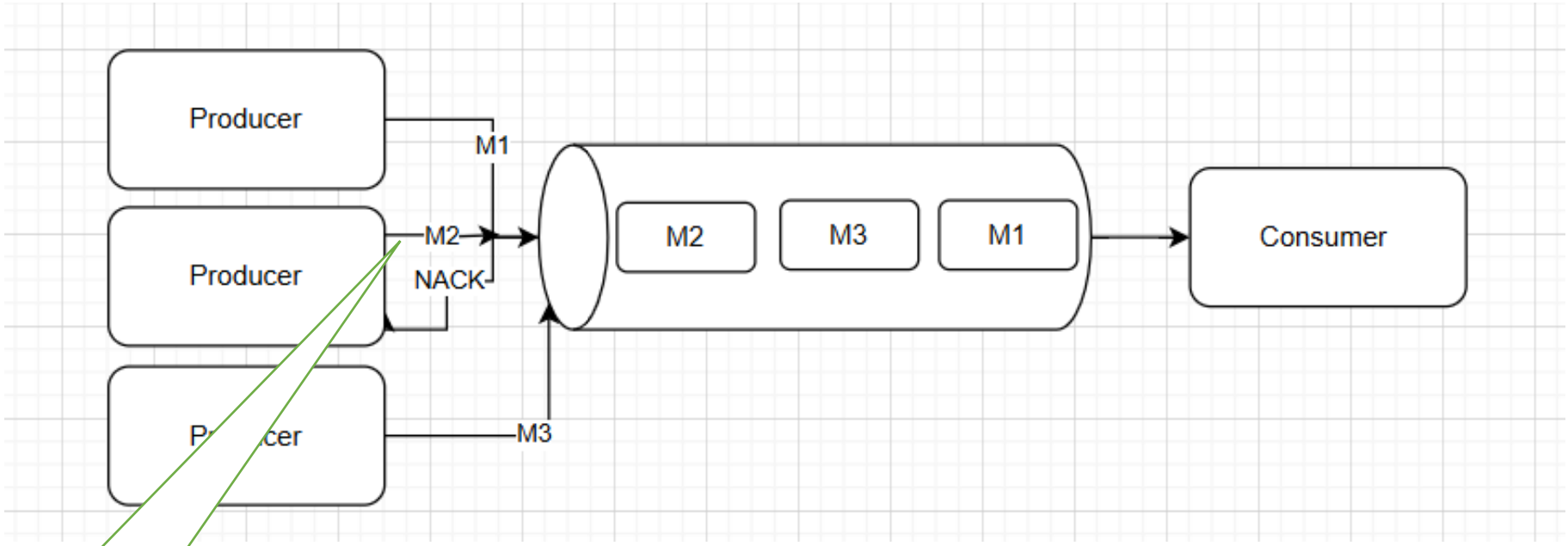


MESSAGE ORDER ERRORS – Description

RabbitMQ ordering issues can arise when:

- **Multiple Consumers**: Messages may arrive in any order because they’re distributed across consumers.
- **Message Redelivery**: If a message is NACKed (not acknowledged) or requeued, it may be processed out of order.
- **Queue Failover and Rebalancing**: Cluster failover and rebalancing may change the order of messages in transit.

MESSAGE ORDER ERRORS – Description



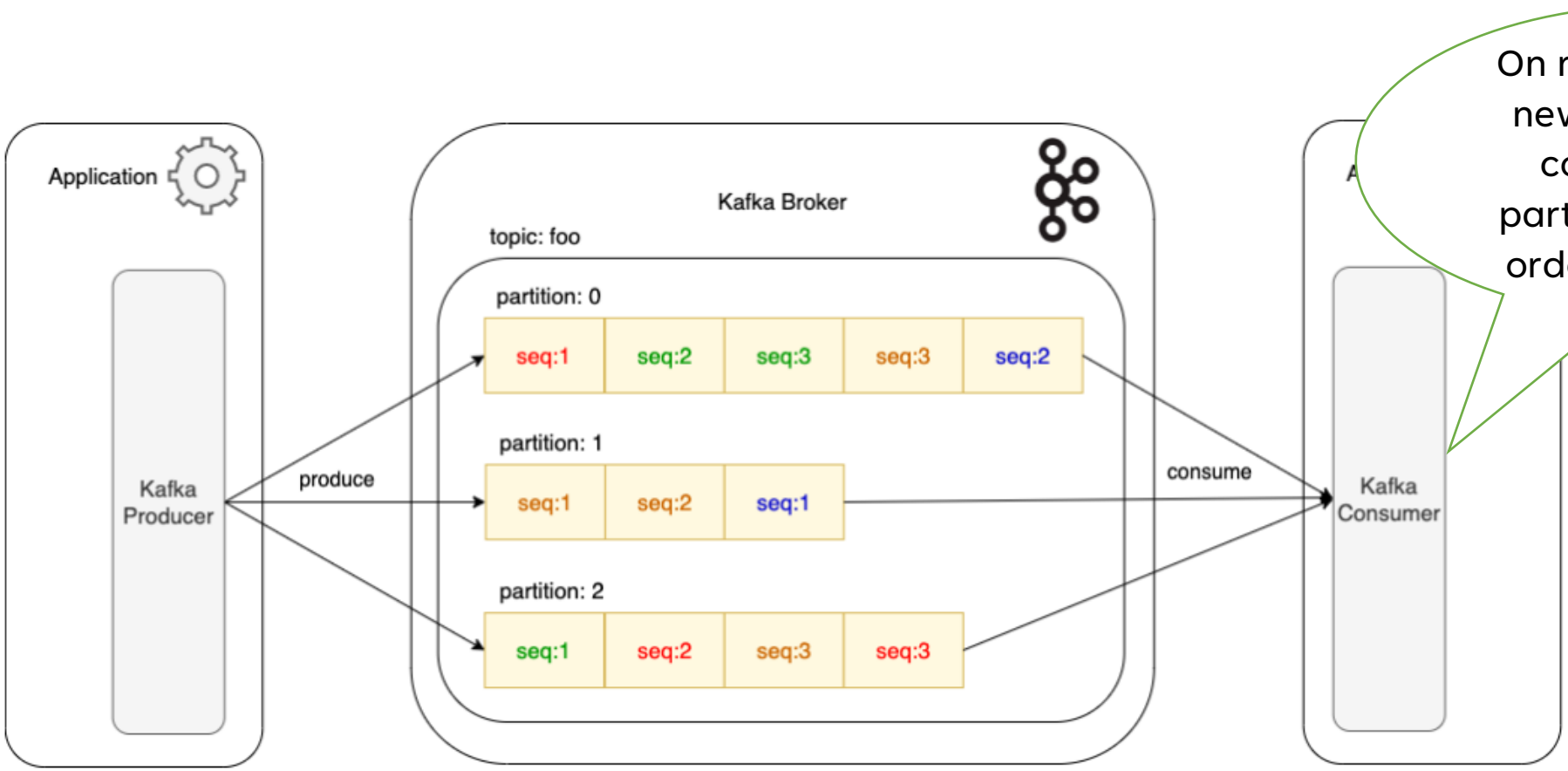
M2 is resent after the NACK. The order changed!

MESSAGE ORDER ERRORS – Description

2. Kafka ordering issues can arise when:

- **Multiple Partitions**: Messages in different partitions of a topic may be out of order relative to each other.
- **Rebalancing**: Consumer group rebalancing may change the partition assignment, affecting processing order.
- **Network Issues**: Network latency may also delay some messages, disrupting perceived order in high-throughput scenarios.

MESSAGE ORDER ERRORS – Description



MESSAGE ORDER ERRORS – How to detect

How to Detect It In RabbitMQ:

1. **Identify Inconsistent Order in Processing**: If your application logs message IDs or timestamps, you can detect ordering issues by reviewing logs or alerts.
2. **Out-of-Order Processing of Related Messages**: Look for signs where subsequent messages are processed before earlier ones (e.g., transaction step 2 before step 1).

MESSAGE ORDER ERRORS – How to detect

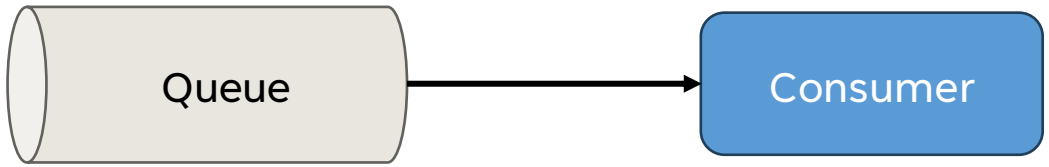
How to Detect It In Kafka:

1. **Track Message Offsets**: Kafka assigns a unique offset to each message within a partition. Track offsets to identify gaps or out-of-order processing.

2. Monitor Processing Logs: Look for inconsistent ordering in logs, where higher offsets are processed before lower ones within a partition.

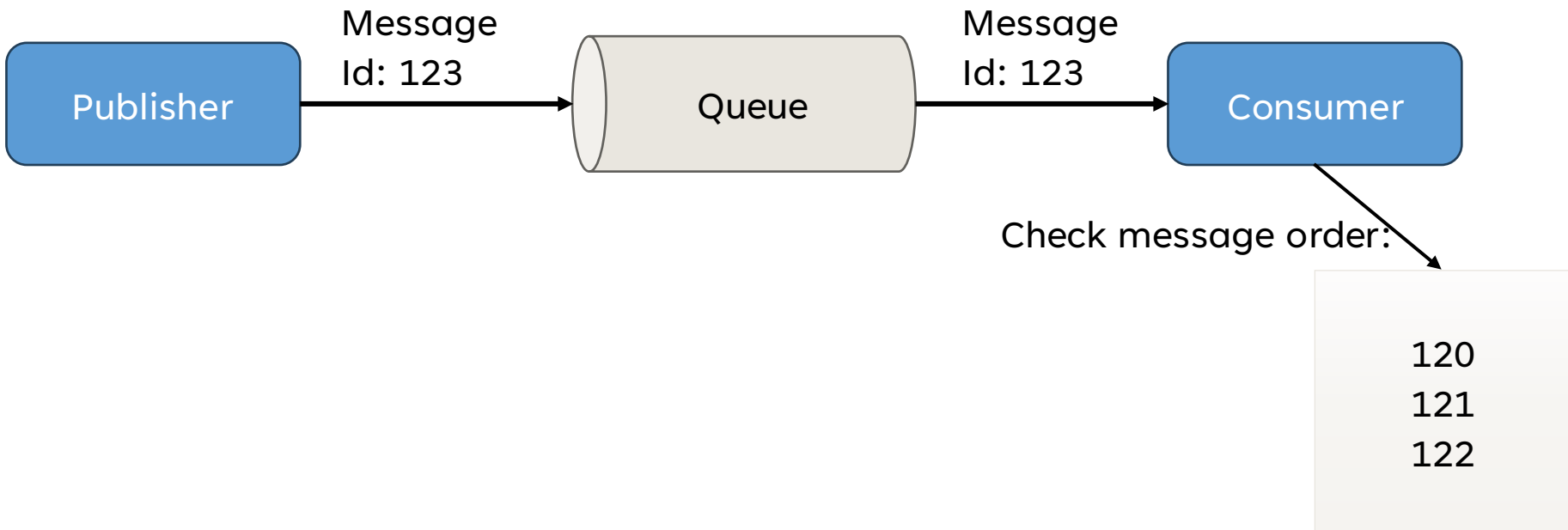
MESSAGE ORDER ERRORS – How to Solve in RabbitMQ

1. **Single Consumer per Queue**: By limiting each queue to a single consumer, you can maintain the order of messages for that queue. However, this approach limits scalability.



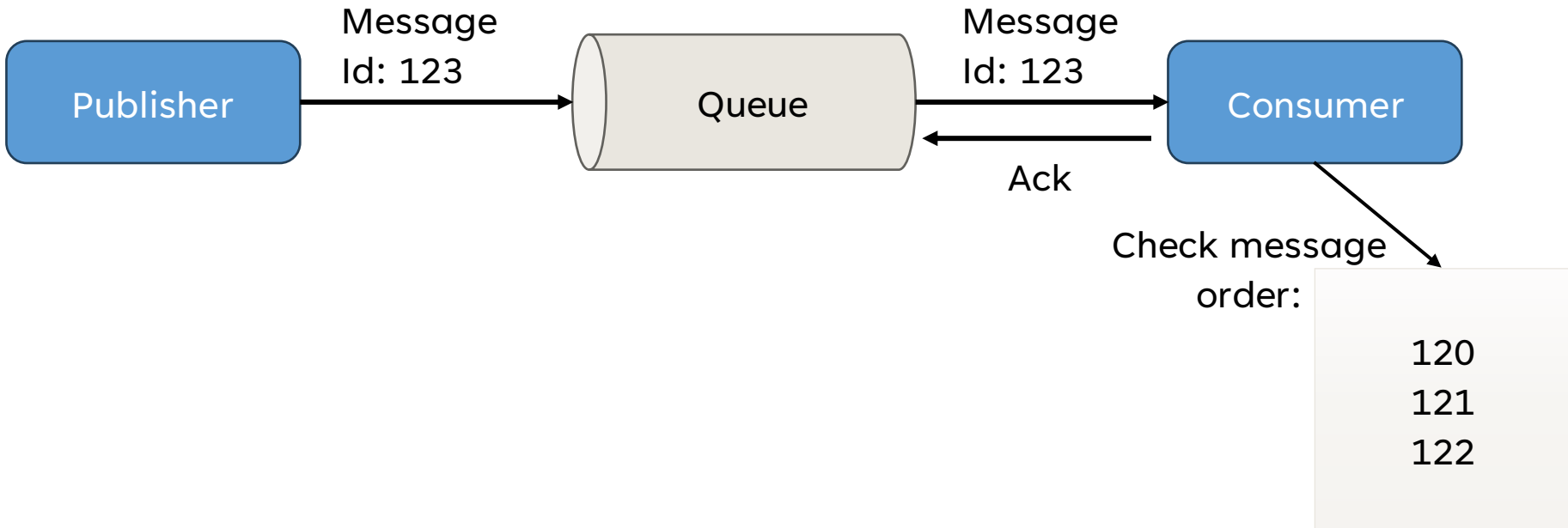
MESSAGE ORDER ERRORS – How to Solve in RabbitMQ

2. Message Sequencing: Add a sequence number to each message so that the consumer can detect and reorder messages if necessary.



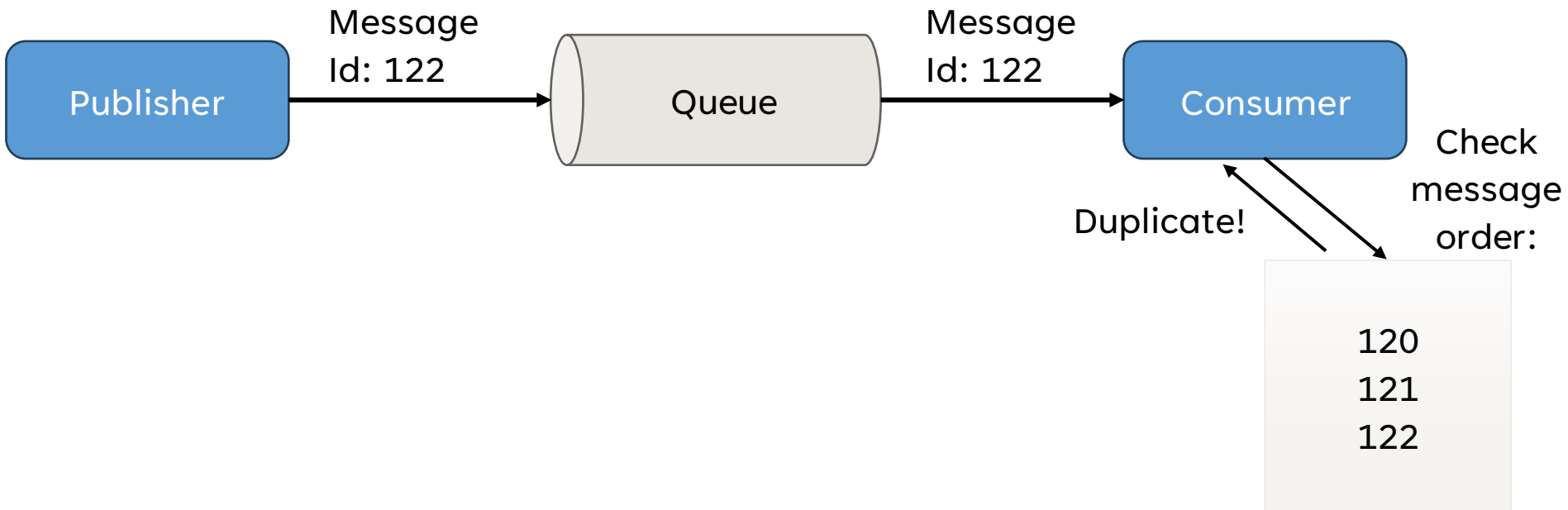
MESSAGE ORDER ERRORS – How to Solve in RabbitMQ

3. Manual Acknowledgments: Implement manual acknowledgment (basic.ack) to ensure the consumer only acknowledges messages after processing them in the correct order.



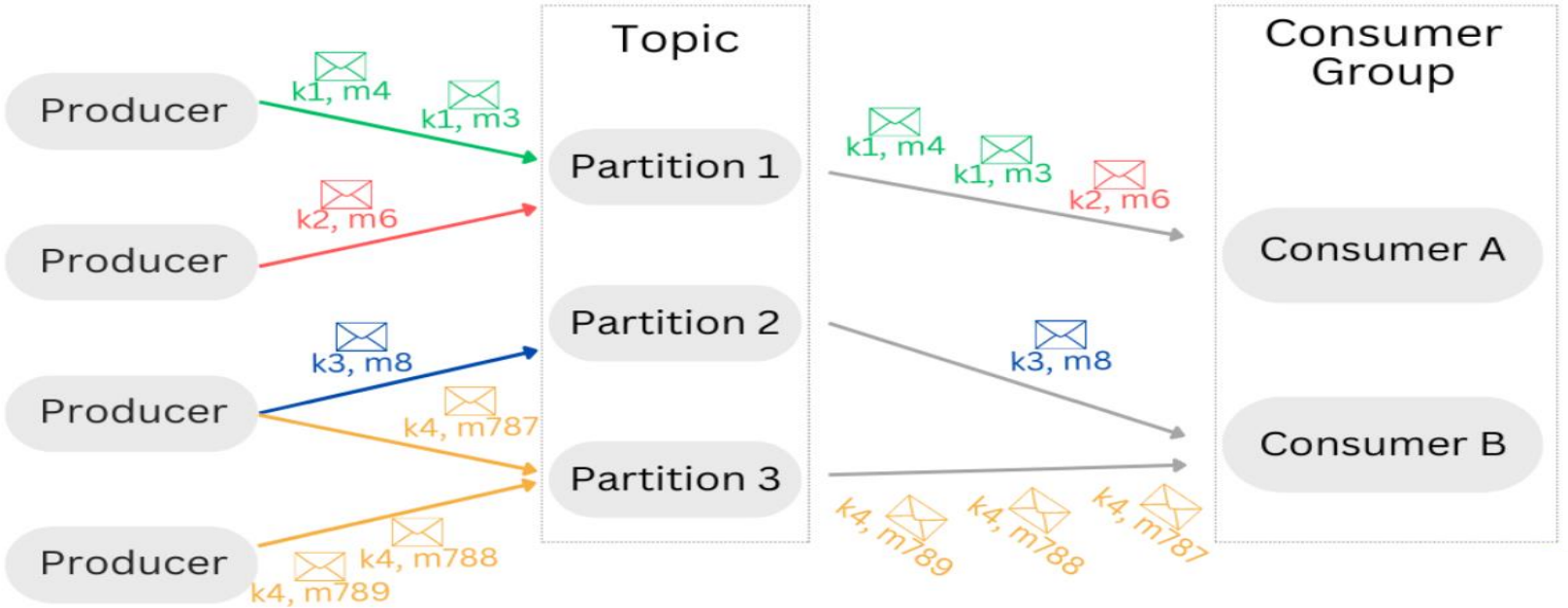
MESSAGE ORDER ERRORS – How to Solve in RabbitMQ

4. Idempotent Processing: Design your application to handle messages idempotently, allowing messages to be processed multiple times without side effects.



MESSAGE ORDER ERRORS – How to Solve in Kafka

1. **Partitioned Ordering**: Use a key to partition messages consistently (e.g., user ID or transaction ID) to ensure related messages land in the same partition and maintain order.



MESSAGE ORDER ERRORS – How to Solve in Kafka

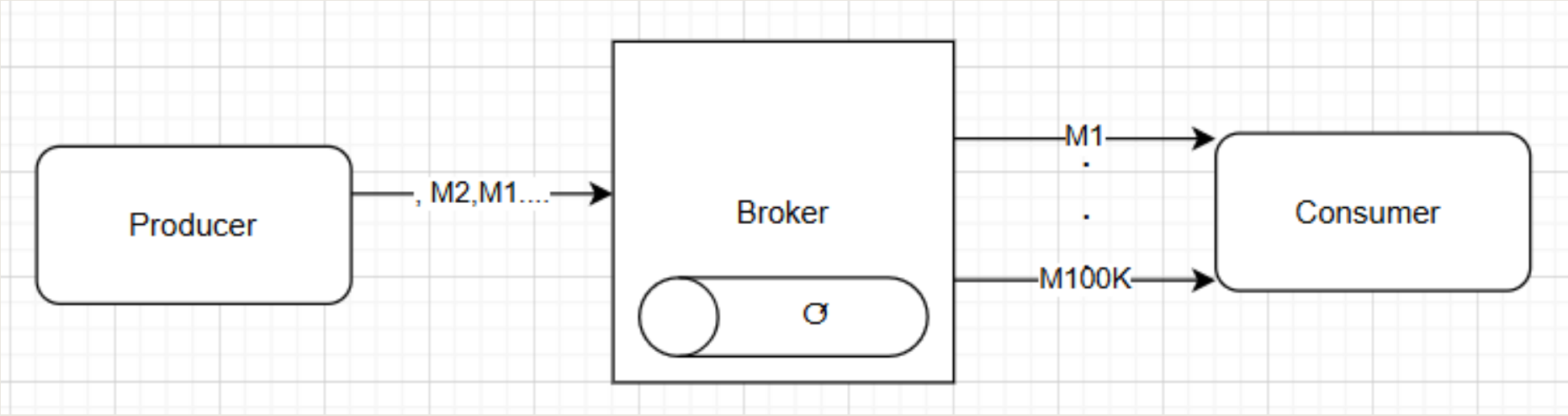
2. Idempotent Processing: Similar to RabbitMQ, design consumers to handle messages idempotently, which minimizes the impact of out-of-order messages.

3. Consumer Group Rebalancing Strategies: Use a sticky partitioning strategy to keep consumers assigned to the same partitions, reducing the likelihood of rebalancing and maintaining order.

TIMEOUTS, CONSUMER LAG, AND BACKPRESSURE

If a consumer takes too long to process messages, it can fall behind, leading to consumer lag.

If the consumer can’t keep up with the rate of incoming messages, it may experience overload.



TIMEOUTS, CONSUMER LAG, AND BACKPRESSURE ERRORS – Description

In RabbitMQ and Kafka, consumers can experience issues like timeouts, lag, and backpressure, especially under high loads or network constraints. These issues affect message processing and can lead to delayed responses or lost messages if not managed properly.

TIMEOUTS, CONSUMER LAG, AND BACKPRESSURE ERRORS – Description

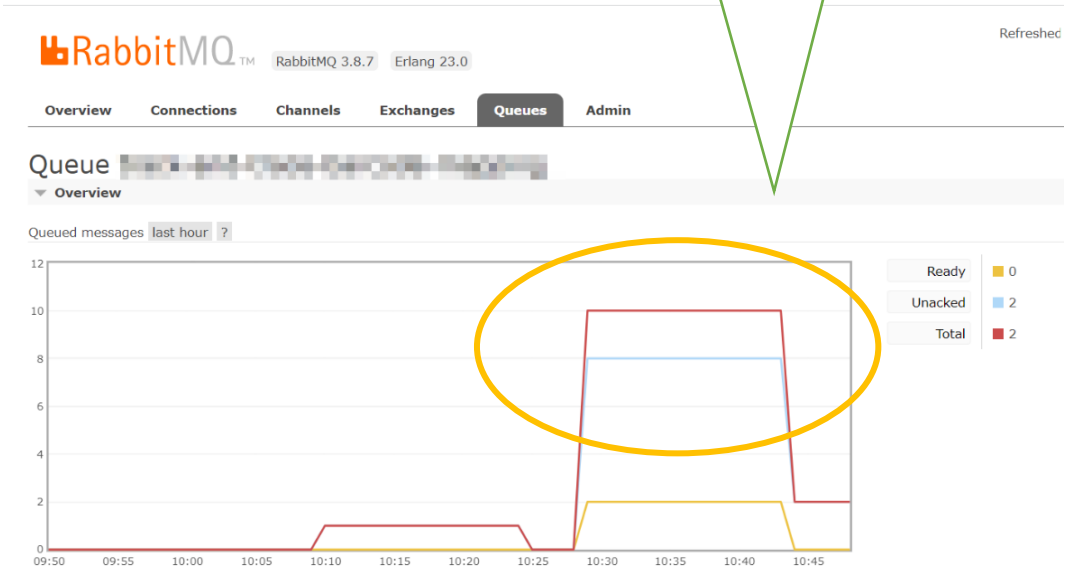
- **Timeouts**: Occur when a consumer fails to acknowledge a message or complete processing within a defined period.
- **Consumer Lag**: Represents the delay between when a message is produced and when it is consumed.
- **Backpressure**: Arises when producers send messages faster than consumers can process, leading to message build-up.

TIMEOUTS, CONSUMER LAG, AND BACKPRESSURE ERRORS – How to detect

1. Monitoring tools: Kafka and RabbitMQ will log timeouts, lag (each platform in a different way) and other useful metrics.

2. Monitor message rate: if the unacknowledged messages number only increases it indicates that the consumer can't keep up.

We are worried when there is a long period of time where the unacked messages are high



TIMEOUTS, CONSUMER LAG, AND BACKPRESSURE ERRORS – How to Solve in RabbitMQ

Timeouts:

1. **Acknowledge Messages Manually**: Use manual acknowledgments (autoAck: false) and send an ack only after processing each message. This prevents RabbitMQ from assuming a message is unprocessed.

2. **Increase Timeout Settings**: Increase the consumer_timeout if the processing is lengthy or unpredictable.

3. **Adjust Heartbeat Interval**: Set an appropriate heartbeat interval on both the RabbitMQ server and client to avoid premature disconnections.

TIMEOUTS, CONSUMER LAG, AND BACKPRESSURE ERRORS – How to Solve in RabbitMQ

Consumer Lag and Backpressure:

1. **Scale Consumers**: Add more consumers to process the message load faster.

2. **Optimize Consumer Logic**: Reduce message processing time by improving code efficiency.

3. **Prefetch Limit**: Set an appropriate prefetch_count to control the number of messages sent to the consumer, preventing it from being overwhelmed.

TIMEOUTS, CONSUMER LAG, AND BACKPRESSURE ERRORS – How to Solve in Kafka

Timeouts:

1. **Increase session.timeout.ms**: Adjust session.timeout.ms to give consumers more time to process messages.

2. **Adjust max.poll.interval.ms**: Ensure this interval covers the maximum processing time required by your consumer. If exceeded, Kafka assumes the consumer is down and reassigns partitions.

3. **Send Regular Heartbeats**: Use the poll() function at regular intervals to keep the consumer active.

TIMEOUTS, CONSUMER LAG, AND BACKPRESSURE ERRORS – How to Solve in Kafka

Consumer Lag and Backpressure:

1. **Scale Consumers**: Add more consumers to process the message load faster.

2. **Parallelize Processing**: Use parallel processing in the consumer to handle multiple messages concurrently.

3. **Optimize Code**: Reduce the time required to process each message.